

GPSDO v1.06 — Kompletna instrukcja obsługi (od zera do locka)

English | **Polski** | Español

 PDF: English · Polski · Español

 Strona projektu · README · Changelog · Tuner

Firmware: GPSDO v1.06-rtos, autor jmnlabs (na bazie oryginalnego GPSDO André Balsy, pętli PI Larsa Waleniusa, akumulatora wielopoziomowego Alana Cashina i autorskiego filtru Kalmana) **Płytki:** WeAct BlackPill STM32F411CE · **OCXO:** 10 MHz (np. Vectron C4550) **GPS:** odbiornik u-blox na Serial1 (LEA-M8T / NEO-M8T / ZED-F9T lub NEO-6M/7M)

Autor: Jarosław Marek Niewiński (jmnlabs) **Asystenci:** Claude Opus 5 (Anthropic) · GLM-5.3 Max (Z.ai) · Qwen3.8-Max — polska i hiszpańska wersja tego podręcznika jest autorstwa GLM-5.3 Max Ta instrukcja nie zakłada **niczego**. Jeśli nigdy nie kompilowałeś firmware, nie wgrywałeś go do mikrokontrolera i nie używałeś terminala szeregowego — zacznij od Części 1 i rób dokładnie to, co tam napisano, po kolei. Każdy krok mówi, co wpisać i co powinieneś zobaczyć. Nic ważnego nie zostawiono jako „zadanie dla czytelnika”.

Streszczenie w 60 sekund: zainstaluj Arduino IDE z rdzeniem STM32 (nie 3.0.0), otwórz szkic, wybierz swój wyświetlacz w `gpsdo_config.h`, kompiluj z *Newlib Nano + Float printf*, wgraj przez ST-Link lub DFU. **Nigdy nie rób pełnego wymazania chipu (Erase Chip)** — twoje ustawienia i kalibracja żyją w sektorze 7 i tylko pełne wymazanie może je zniszczyć. Potem, po serialu na 115200: CT, poczekaj, LC, poczekaj, LA 12, SAW 1, ES. Patrz, jak faza trzyma się zera. Gotowe.

Spis treści

- Część 1 — Kompilacja firmware
- Część 2 — Wgrywanie i zasada sektora 7
- Część 3 — Pierwsze uruchomienie: kalibracja, algorytm, zapis
- Część 4 — Czternaście algorytmów (0–13) i ich parametry
- Część 5 — Wyświetlacze: co znaczy każde pole
- Część 6 — Telemetria serialowa (raport 6-liniowy)
- Część 7 — Spis komend (wszystkie)
- Część 8 — Tuner na PC
- Część 9 — Ustawienia, flash ring i zużycie
- Część 10 — Diagnostyka problemów
- Aneks A — Jak działa GPSDO, prostymi słowami
- Aneks B — PID dla opornych
- Aneks C — Słowniczek

Część 1 — Kompilacja firmware

1.1 Co trzeba zainstalować

- 1. **Arduino IDE** (1.8.x lub 2.x — oba działają).
- 2. **Rdzeń STM32duino w wersji 2.2.0 do 2.12.0.** Boards Manager → szukaj „stm32” → **STM32 MCU based boards by STMicroelectronics**. Zainstaluj 2.12.0 (ostatnia wspierana).

UWAGA — rdzeń 3.0.0 nie działa. Wyszedł w lipcu 2026 i łamie ten projekt na trzy sposoby: `1toa()` już nie istnieje (błąd kompilacji), `HardwareSerial Serial12(PA3, PA2)` się nie kompiluje, a `TFT_eSPI` przestaje sterować panelami (trwały biały ekran). Trzymaj 2.12.0 lub starszy, dopóki projekt nie ogłosi inaczej.

1. **Biblioteki** (Library Manager, instaluj po dokładnej nazwie):

Biblioteka	Po co
STM32duino FreeRTOS	system operacyjny
TinyGPS++	parsowanie zdań GPS
U8g2	wyświetlacze OLED (tylko gdy jakiś włączysz)
Adafruit AHTX0	czujnik temperatury/wilgotności AHT10/AHT20
Adafruit BMP280	czujnik ciśnienia/temperatury BMP280
Adafruit INA219	monitor napięcia/prądu zasilania
hd44780	LCD znakowy 20x4 (tylko gdy włączysz)
TFT_eSPI	wyświetlacz TFT na SPI

1. Nic więcej. Nie ma biblioteki EEPROM — ustawienia żyją we flashu wbudowanym (Część 2).

1.2 Otwarcie szkicu

Rozpakuj projekt. Otwórz `GPSDO_FreeRTOS/GPSDO_FreeRTOS.ino` w Arduino IDE. Folder musi zachować nazwę — Arduino wymaga, żeby nazwa folderu zgadzała się z plikiem `.ino`.

1.3 Dobór sprzętu w `gpsdo_config.h`

Ten jeden plik decyduje, co się skompiluje. Linie aktywne zaczynają się od `#define`; wyłączone od `//`. Odkomentuj dokładnie to, co masz na płycie, zakomentuj to, czego nie masz. Konfiguracja fabryczna to w pełni obsadzona płytką (duży TFT + wszystkie czujniki); dopasuj ją do rzeczywistości.

Wyświetlacz główny — wybierz dokładnie jeden:

Flaga	Sprzęt
<code>GPSDO_TFT_ILI9488</code>	SPI TFT 320x480 (domyślny; pracuje w poziomie 480x320)
<code>GPSDO_TFT_ST7789</code>	SPI TFT 240x320
<code>GPSDO_TFT_ILI9341</code>	SPI TFT 240x320
<code>GPSDO_OLED_SH1106</code> / <code>_SSD1306</code> / <code>_SSD1309</code>	OLED 128x64 na I2C (tylko jeden)
<code>GPSDO_LCD_20x4</code>	HD44780 20x4 przez PCF8574 na I2C
<code>GPSDO_TM1637</code> / <code>GPSDO_TM1637_6</code>	zegar LED 4-cyfrowy / 6-cyfrowy (wzajemnie wyłączone, i nie razem z LCD)
<code>GPSDO_HT16K33</code>	zegar LED 4-cyfrowy na I2C (domyślnie włączony; niezależny od wyświetlacza głównego)

Czujniki i dodatki (fabrycznie włączone — zakomentuj, czego nie masz):

Flaga	Sprzęt
GPSDO_AHT10	AHT10/AHT20 na I2C
GPSDO_BMP280_I2C	BMP280 na I2C
GPSDO_INA219	INA219 na I2C
GPSDO_VCC / GPSDO_VDD	pomiar szyny 5 V / 3,3 V
GPSDO_UBX_CONFIG	ustawienie NEO-6M/7M w tryb binarny przy starcie
GPSDO_FAKE_UBLOX	chiński klon u-bloxa: tylko proba baudrate, zero konfiguracji UBX (patrz Część 3.2)
GPSDO_GPS_TIMING	obsługa odbiorników timingowych (LEA-6T/M8T/NEO-M8T/ZED-F9T): survey-in, qErr
GPSDO_PICDIV	wyjście zbrojenia dzielnika picDIV na PB3
GPSDO_LTIC	sprzętowy detektor fazy (ramp TIC na PA1) — wymagany dla algorytmów 10, 11, 12. Włączaj TYLKO gdy detektor jest fizycznie zamontowany: bez niego PA1 pływa, a pętla dyscyplinuje OCXO szumem ADC
GPSDO_LTIC_ACTIVE_RESET	wariant detektora z aktywnym rozładowaniem kondensatora (wyłączony dla klasycznego detektora RC Kaashoeke)
GPSDO_PWM_DITHER	24-bitowe napięcie sterujące z ditherowanego PWM 13-bit (nie wyłączaj — to ten dobry DAC)
GPSDO_DAC_EXT	zewnętrzny DAC AD5680, 18-bit, bit-bang na PB4/PB0/PB2 — wyklucza się z GPSDO_PWM_DITHER
GPSDO_GEN_2kHz_PB5	prostokąt testowy 2 kHz na PB5
GPSDO_BLUETOOTH	moduł HC-06 na PA2/PA3
GPSDO_BLUETOOTH_PARALLEL	USB i Bluetooth równocześnie (domyślnie włączony)

Plik odmawia kompilacji (celowo), jeśli wybierzesz dwa wyświetlacze tego samego typu albo dwa elementy kłócące się o te same piny. Przeczytaj komunikat `#error` — nazywa konflikt.

Zostaw **włączone** `GPSDO_LTIC` i `GPSDO_PWM_DITHER`, chyba że naprawdę nie masz sprzętu detektora: bez LTIC tracisz algorytmy 10–13, czyli cały sens v1.06.

1.4 Konfiguracja TFT_eSPI (tylko TFT)

TFT_eSPI konfiguruje się w bibliotece, nie w szkicu. Znajdź `Arduino/libraries/TFT_eSPI/User_Setup.h` i umieść w nim blok dla swojego panelu:

240x320 (ST7789 lub ILI9341):

```
#define ST7789_DRIVER           // or ILI9341_DRIVER
#define TFT_WIDTH  240
#define TFT_HEIGHT 320
#define TFT_MISO  PA6           // required on STM32 even if the display has no MISO pin
#define TFT_MOSI  PA7
#define TFT_SCLK  PA5
#define TFT_CS    PB13
#define TFT_DC    PB12
#define TFT_RST   PB15
#define TFT_RGB_ORDER TFT_BGR
#define TFT_INVERSION_OFF       // fixes inverted colours on some ST7789 modules
#define LOAD_GLCD               // font 1 – frequency readout + splash
```

```
#define LOAD_FONT2          // font 2 – header + data grid
#define LOAD_FONT4          // font 4 – status bar, busy messages
#define SPI_FREQUENCY 4000000 // 40 MHz works; drop to 27 MHz for long wires
```

320x480 (ILI9488 / ILI9486):

```
#define ILI9488_DRIVER      // works for both ILI9488 and ILI9486
#define TFT_WIDTH 320
#define TFT_HEIGHT 480
// same TFT_MISO/MOSI/SCLK/CS/DC/RST/RGB_ORDER lines as above
#define LOAD_GLCD          // font 1 – splash credits
#define LOAD_FONT4         // font 4 – splash subtitle path
#define LOAD_GFXFF         // REQUIRED on this panel – free fonts
#define SPI_FREQUENCY 4000000 // don't skimp: this panel pushes 2.4x the pixels
```

Podłączenie panelu: SCK→PA5, SDI→PA7, RES→PB15, D/C→PB12, CS→PB13. `TFT_MISO` PA6 musi być zdefiniowane, nawet jeśli nic do niego nie jest podłączone.

1.5 build_opt.h — zestaw w spokoju

Plik zawiera trzy flagi kompilatora i nie wymaga od Ciebie żadnej akcji:

```
-DSERIAL_RX_BUFFER_SIZE=256 -DSERIAL_TX_BUFFER_SIZE=512
-DCDC_TRANSMIT_QUEUE_BUFFER_PACKET_NUMBER=16
```

Pierwsza linia powiększa bufory seriala, żeby zdania GPS nie ginęły. Druga rozszerza kolejkę nadawczą USB-CDC ze 128 bajtów do 1 KB, dzięki czemu raport telemetry 1 Hz (około 400 znaków) zawsze się mieści — bez tego host, który podłączy się i nie czyta, mógłby wstrzymać raport na sekundy. Plik jest pobierany automatycznie przez rdzeń STM32. Nie usuwaj go.

1.6 Menu Tools — płytki i biblioteka uruchomieniowa

W Arduino IDE: **Tools** →

- **Board:** "Generic STM32F4 series" → **BlackPill F411CE**.
- **C Runtime Library:** Newlib Nano + Float Printf/Scanf. To obowiązkowe. Bez tego firmware się skompiluje, ale wypisze śmieci tam, gdzie powinna być liczba zmiennoprzecinkowa (wszystkie napięcia, częstotliwości i fazy).

1.7 Włącz USB CDC (żeby płytka dała konsolę po USB)

CDC sprawia, że `Serial` staje się wirtualnym portem COM na kablu USB. Banner startowy, wiersz poleceń i raport 1 Hz — wszystko tym jedzie. Bez CDC płytka nadal działa — wyświetlacz się odświeża, pętla dyscyplinuje oscylator — ale po USB wygląda na całkiem martwą: brak banera, brak znaku zachęty, nic.

1. **Tools** → **USB support** (lub "USB support (if available)") → "CDC (generic Serial supersedes U(S)ART)". Dokładna nazwa różni się nieco między wersjami rdzenia; wybierz tę, która wspomina **CDC** i **generic Serial**.
2. Skompiluj ponownie i wgraj. Zmiana tego ustawienia zmienia firmware — to nie jest przełącznik działający w locie.
3. Po wgraniu **naciśnij raz przycisk RESET na płycie**. Chip przełącza się z urządzenia USB DFU (wgrywanie) na urządzenie CDC serial; bez resetu część hostów rozmawia dalej ze starym, już nieistniejącym urządzeniem.
4. Pojawia się nowy port COM (Windows: "STM32 Virtual COM Port", **VID 0483, PID 5740**). Jeśli Windows pokazuje "Unknown device" albo "device descriptor request failed", zainstaluj raz sterownik przez Zadig — patrz też Część 2.5.
5. Otwórz port na 115200. Firmware **czeka do 3 sekund** po resecie, aż host otworzy port, zanim wydrukuje banner — wolny sterownik nie połka pierwszych linii. Jeśli otwierasz terminal i nic nie widzisz, naciśnij RESET jeszcze raz przy już otwartym terminalu.

Dwie rzeczy warte zapamiętania:

- Tryb DFU (wgrywanie) i tryb CDC (praca) to dla Windows **dwa różne urządzenia USB**. Pierwsze cykle wgrywania mogą każdy raz odpalić rundę instalacji sterownika — to normalne, po chwili się uspokaja.

- Przy skompilowanym `GPSDO_BLUETOOTH_PARALLEL` (domyślna konfiguracja) wszystko, co widać po USB, jest dublowane na UART Bluetooth przy **57600** — gotowa zapasowa konsola, gdyby USB kiedyś stawiało opór.

1.8 Kompilacja i kontrola rozmiaru

Sketch → Verify/Compile. Po zakończeniu przeczytaj linię `Sketch uses NNNNNN bytes`.

NNNNNN musi zostać poniżej 393216. To 384 KB — wszystko powyżej należy do pierścienia ustawień w sektorze 7 (Część 2). Zignoruj procent, który wypisuje IDE: liczony jest względem pełnych 512 KB chipu, więc build, który „mieści się” procentowo, może przekroczyć granicę sektora 7. Samo v1.06 zajmuje około 250 KB — zapas duży, ale nie ignoruj nagłego wzrostu.

Który wsad naprawdę działa? Baner wypisuje czas kompilacji — a ten czas należy do **szkicu**, nie do firmware'u. Builder Arduino nie przekompilowuje jednostki, której źródła się nie zmieniły, więc zmiana `GPSDO_algorithms.cpp` i wgranie zostawia plik obiektowy szkicu nietknięty, ze starszą datą w środku. Baner wypisuje też `image CRC32`, liczone przy starcie z samej pamięci: nie wyjdzie z cache'a buildera, zmienia się przy zmianie dowolnego bajtu dowolnej jednostki kompilacji, a dwie płytki z tym samym wsadem pokażą tę samą liczbę. Gdy log i pamięć się nie zgadzają, wierz tej liczbie. `v` pokazuje ją w każdej chwili.

Jeśli stempel czasu też ma być prawdziwy, uruchom przed kompilacją `python3 tools/bumpbuild.py` — dotyka `build_id.h`, który szkic dołącza wyłącznie w tym celu, a to zmusza builder do przekompilowania szkicu i tylko szkicu. Numer pojawia się w banerze jako `build N`.

Część 2 — Wgrywanie i zasada sektora 7

2.1 Jak podzielony jest flash 512 KB

Sektory	Zakres adresów	Zawartość
0 – 5	0x08000000 – 0x0803FFFF	Twój firmware (384 KB)
6	0x08040000 – 0x0805FFFF	zapas (bufor wzrostu)
7	0x08060000 – 0x0807FFFF	flash ring: ustawienia + kalibracja + dane wyuczone

W sektorze 7 mieszka wszystko, czego urządzenie nauczyło się o samym sobie:

- twoje zapisane ustawienia (wybór algorytmu, wszystkie parametry pętli, strefa czasowa),
- kalibrację czułości oscylatora z `CT`,
- kalibrację detektora fazy z `LC`,
- wyuczony model dryfu/tłumienia i ostatni punkt pracy.

Odtworzenie tego wszystkiego kosztuje godzinę na stole. Ochrona sektora 7 jest więc **najważniejszą zasadą eksploatacyjną całego projektu**.

2.2 Złota zasada

Zwykle wgrywanie nigdy nie dotyka sektora 7. Pełne wymazanie chipu go niszczy.

- Upload z Arduino IDE, DFU i bootloader STM32duino przeprogramowują tylko sektory 0–5. Ustawienia przeżywają każdą rutynową aktualizację firmware.
- „Erase Chip” w ST-LINK Utility / STM32CubeProgrammer albo `erase` bez zakresu adresów w J-Link wymazuje cały chip **razem z sektorem 7**. Nigdy nie używaj tego na działającej, skalibrowanej jednostce.

Jeśli wgrywasz ręcznie przez ST-Link lub J-Link, wymazuj **wyłącznie** zakres firmware i dopiero wtedy ładuj:

```
> erase 0x08000000 0x0803FFFF
> loadbin firmware.bin 0x08000000
```

Jeśli sektor 7 kiedyś zginie: nic się nie psuje. Przy następnym starcie firmware wykrywa pusty pierścień, formatuje go i startuje z wartościami domyślnymi — po prostu ponawiasz `CT`, `LC`, ustawiasz algorytm i strefę czasową, i `ES`. Tracisz strojenie, nie urządzenie.

Przed pierwszym wgrywaniem na skończoną jednostkę zrób pełny backup (przykład J-Link; ST-LINK Utility ma odpowiedni przycisk "save"):

```
> JLinkExe -device STM32F411CE -if SWD -speed 4000
> savebin backup_full.bin 0x08000000 0x80000
> exit
```

Przywrócenie: `loadbin backup_full.bin 0x08000000`.

2.3 Wgrywanie metodą A — ST-Link (najprościej)

1. Podłącz ST-Link V2 do pinów SWD BlackPilla (SWDIO, SWCLK, GND, 3V3).
2. Arduino IDE → Tools → **Upload method: "STM32CubeProgrammer (SWD)"**.
3. Naciśnij Upload. Gotowe. Programator zapisuje tylko to, co zajmuje szkielet.

2.4 Wgrywanie metodą B — DFU po USB (bez programatora)

Jaki BOOT0 masz na płytce? Starsze BlackPille WeAct mają zworkę BOOT0; aktualne (v3.1) mają tylko przycisk BOOT0. Użyj przepisu dla swojego wariantu.

Płytką ze zworką: przełóż zworkę BOOT0 na 1, naciśnij RESET — płytka pojawi się jako "STM32 BOOTLOADER". Po uploadzie wróć zworką do 0 i naciśnij RESET.

Płytką z przyciskiem (v3.1), niezawodnie: odłącz zasilanie zewnętrzne; **naciśnij i trzymaj BOOT0**, podłącz kabel USB **trzymając przycisk**; po kilku sekundach puść BOOT0 — Windows nie powinien zgłosić "nieznane urządzenie", a STM32CubeProgrammer widzi płytkę. Wgraj, potem naciśnij RESET. (Trzymanie BOOT0 z jednoczesnym stukaniem NRST, jak podają inne źródła w internecie, zwykle **nie** działa — zwłaszcza gdy płytka jest już wlutowana w PCB.)

Windows może pierwszy raz poprosić o sterownik — patrz uwaga niżej.

2.5 Po każdym wgrywaniu: uwaga o USB

- Po zakończeniu uploadu **naciśnij przycisk RESET na płytce** — urządzenie USB czysto przeenumeruje się z trybu bootloadera na CDC serial firmware.
- USB serial firmware to **VID 0483, PID 5740** ("STM32 Virtual COM Port"). Jeśli Windows odmawia otwarcia portu, zainstaluj raz sterownik przez Zadig.

Część 3 — Pierwsze uruchomienie: kalibracja, algorytm, zapis

Potrzebny jest program terminala (PuTTY, Tera Term, monitor serialowy Arduino IDE albo tuner z Części 8 — tuner jest najwygodniejszy).

3.1 Połączenie

- 1. Podłącz USB (i/lub Bluetooth na 57600 — w konfiguracji fabrycznej oba działają równocześnie).
- 2. Otwórz port na 115200, 8N1. Końce linii: CR lub LF — oba działają.
- 3. Naciśnij **Enter**. Powinieneś zobaczyć znak zachęty albo start raportu 1 Hz.
- 4. Wpisz **H** i **Enter**. Przewinie się pełna lista komend — to Część 7 tej instrukcji, na żywo.

Log startowy linia po linii. Pierwsze dziesięć sekund po resecie wypisuje stałą sekwencję. Oto typowa z działającej, już skonfigurowanej jednostki (dokładne linie zależą od sprzętu), a dalej — co każda znaczy:

```
=====
GPSDO v1.06-rtos compiled 2026-08-23 14:32:45
FreeRTOS port by J. M. Niewinski with Claude, GLM-5.3 Max & Qwen3.8-Max AI
https://github.com/jmnlabs/GPSDO_FreeRTOS
Inspired by GPSDO v0.06c by Andre Balsa
https://github.com/AndrewBCN/STM32-GPSDO
Algos 0-2 original design by Andre Balsa
Algos 3-9 by J. M. Niewinski
Algo 10 (LTIC 3-stage) inspired by Dan Wiering's measurements
Algo 11 (LTIC-Lars) after Lars Walenius' PI loop
Algo 12 (multi-level accumulator) after Alan Cashin (MIS42N)
Algo 13 (Kalman filter) by J. M. Niewinski - original to this project
Type H = help SW = stack diagnostics
=====
Reset cause: POWER-ON/BROWN-OUT PIN/NRST
Flash ring: sector 7 ready
Settings: recalled from flash ring
Live store: LRN + LC applied from flash ring
Initial PWM=44653 algo=12 time_offset_min=120
GPS init: probing baud rate...
GPS detected at 38400
GPS: disabling noisy NMEA at 38400 baud
GPS: 4/4 NMEA sentences disabled
GPS: sending UBX config at 38400 baud
UBX: CFG-NAV5 ACK
LEA-T: accepted CFG-TMODE2 (28B)
Hardware configured, creating RTOS objects...
Timers started
Starting FreeRTOS scheduler
HW: AHT10/AHT20 sensor OK (I2C 0x38)
HW: BMP280 sensor OK (I2C 0x77)
HW: INA219 sensor OK (I2C 0x40)
HW: LTIC phase input enabled (PA1) - needs the ramp detector hw
TFT: init start (SPI1 PA5/PA7, CS=PB13 DC=PB12 RST=PB15)
TFT: freq-band sprite (4-bit) created
TFT: header sprite (4-bit) created
TFT: data sprite (1-bit) created
```

Potem startuje raport 1 Hz (Część 6). Co mówi każda linia:

Linia	Znaczenie
banner (GPSDO v1.06-rtos ...)	tożsamość firmware. Jeśli zamiast tego widzisz śmieci: zła prędkość — użyj 115200

Linia	Znaczenie
Reset cause:	dlaczego chip się zrestartował. POWER-ON/BROWN-OUT PIN/NRST = normalne włączenie zasilania lub przycisk reset. SOFTWARE = reset zarządzany przez firmware (RB, koniec uploadu). INDEP-WDG/WINDOW-WDG = watchdog (ten firmware żadnego nie używa — traktuj jako sygnał awarii). Dodatkowa linia -> supply dipped: check the 3V3 rail under load pojawia się po brown-out: szyna 3V3 siadła pod obciążeniem OCXO — napraw zasilanie, nie ignoruj tego
Flash ring: sector 7 ready	pierścień ustawień w sektorze 7 znaleziony i poprawny
Flash ring: sector 7 blank/formatted (defaults)	pusty lub wymazany pierścień — normalne przy pierwszym wgrywaniu ; firmware go formatuje i używa wartości domyślnych z kompilacji
Settings: recalled from flash ring	zapisane ustawienia (algorytm, parametry pętli, strefa czasowa, flagi) zostały zastosowane
Settings: none stored (compile-time defaults)	nic jeszcze nie zapisano — oczekiwane na nowej jednostce; przejdź Część 3 i wykonaj ES
Live store: LRN + LC applied from flash ring	dane wyuczone (kalibracja LC, model dryfu, ostatni punkt pracy) zastosowane — linia pojawia się dopiero na jednostce kalibrowanej
Initial PWM=... algo=... time_offset_min=...	punkt pracy wybrany przy starcie: ostateczny kod PWM (dane wyuczone nadpisują ustawienia, gdy są świeższe), przywrócony algorytm i przesunięcie strefy czasowej w minutach
GPS init: probing baud rate... / GPS detected at ...	odbiornik znaleziony, prędkość zmierzona (preferuje 38400)
GPS: no response to baud probe, defaulting to 9600	żaden odbiornik nie odpowiedział — zanim ruszysz dalej, sprawdź okablowanie modułu GPS
GPS: disabling noisy NMEA... / GPS: sending UBX config... / UBX: CFG-NAV5 ACK	odbiornik przechodzi w tryb binarny i dynamikę stacjonarną; warianty NAK/no response są przeżywalne — odbiornik po prostu zachowuje obecną konfigurację
LEA-T: accepted CFG-TMODE2	odbiornik timingowy (klasa LEA-M8T) skonfigurowany w survey-in / Time Mode — pojawia się tylko przy GPSDO_GPS_TIMING i odbiorniku timingowym
Hardware configured... / Timers started / Starting FreeRTOS scheduler	wewnętrzny start; wszystkie trzy zawsze powinny się pojawić
HW: <czujnik> OK (I2C ...) / HW: <czujnik> not found	skan magistrali I2C: każdy czujnik raportuje obecność lub jej brak. Brakujący czujnik nie jest fatalny — jednostka działa bez niego, tylko pole w raportach zostaje puste
HW: LTIC phase input enabled (PA1)	firmware ZBUDOWANO ze ścieżką detektora fazy, a PA1 jest pod nią skonfigurowane — nie odróżnia zamontowanego detektora od pływającego pinu, więc to nie jest test sprzętu (algorytmy 10–13 zależą od prawdziwego detektora)
TFT: init start (...)	start inicjalizacji wyświetlacza. Jeśli po tej linii nic nie następuje , okablowanie TFT albo User_Setup.h jest złe (Część 1.4) — to przypadek „ekran biały/martwy, ale serial żyje”
TFT: ... sprite FAILED – direct-draw fallback	wyświetlacz działa, ale wolniej się przerysowuje (zabrakło RAM na sprity anty-miganiowe) — kosmetyka, nie awaria

Linie pojawiające się **później**, w trakcie pracy, i ich znaczenie:

Linia	Znaczenie
LTIC: running UNCALIBRATED (run LC)	aktywny jest algorytm 10/11/12, ale LC nigdy nie było uruchamiane — liczby fazy są nominalne, nie zmierzone. Uruchom LC
LTIC ACQ: polarity unset – run 'LPOL -1' (or +1)	algorytm 10 odmawia sterowania do ustawienia polaryzacji (bezpiecznie czeka zamiast zgadywać). Algorytmy 11 i 12 zakładały +1 i sterowały mimo to; teraz czekają tak samo
picDIV: armed (output stopped, waiting for 1PPS sync)	dzielnik zbrojony; przerwa 1–1,2 s na jego wyjściu jest oczekiwana, zanim zsynchronizuje się do kolejnego zbocza PPS
LEA-T: ... survey ... % / s) – continuing anyway	monitor survey-in; „continuing anyway” znaczy, że survey osiągnął limit czasu powyżej celu dokładności — zwykle zaczepiony widok nieba
!!! FreeRTOS ... (configASSERT / STACK OVERFLOW / MALLOC FAILED)	firmware złapał awarię, z plikiem/linią albo nazwą taska — niebieska LED miga w tym samym czasie szybko. Zanotuj jedno i drugie; jednostka potrzebuje resetu (RB)

Dziewicza jednostka automatycznie zarządza kalibracją przy pierwszym starcie — możesz zobaczyć odliczanie kalibracji zanim cokolwiek innego.

3.2 Pozwól GPS-owi się ustabilizować

- Daj antenie czysty widok nieba. Parapet działa; geodezyjna antena na dachu działa lepiej.
- Przy odbiorniku timingowym (M8T/F9T...) i skompilowanym `GPSDO_GPS_TIMING` przy starcie działa **survey-in**: odbiornik mierzy własną pozycję przez co najmniej 300 s, dopóki szacunek nie będzie lepszy niż 5 m, po czym przechodzi w **Time Mode** na stałą pozycję. Od tej chwili wyświetlacz pokazuje `HDOP:TIME` zamiast liczby. Survey-in musi się zakończyć tylko raz; powtarza się po zaniku zasilania. (`SV 0` wyłącza, `SV 1` włącza z powrotem, przy następnym starcie.)

Jak zapisać survey na stałe. Survey 300 s / 5 m, który firmware robi przy starcie, jest kompromisem: dość długi, by się przydał, dość krótki, by nikt na niego nie czekał. Jeśli możesz zostawić odbiornik na kilka godzin, zrób survey raz w u-center i zapisz go w podtrzymywanej bateryjnie konfiguracji modułu — u-center **V8.29** ma właściwe komendy (UBX-CFG-TMODE2 uruchamia survey, potem UBX-CFG-CFG → *Save current configuration*). Długi survey daje lepszą pozycję stałą i przeżywa wyłączenia zasilania: firmware sprawdza Time Mode przy starcie i pomija własny survey, gdy odbiornik już go zgłasza. To zalecenie Alana Cashina — najtańsza dokładność w całej konstrukcji. - Rozgrzewka OCXO trwa 300 s (`WU 0` ją pomija; zostaw włączoną). - Żółta LED jest **wyłączona bez fixa GPS, świeci ciągle przy fixie** — gdy coś wygląda źle, zaglądaj tam najpierw.

Jaki moduł GNSS wybrać? Oryginalny odbiornik timingowy u-blox (klasa LEA-M8T) to baza, pod którą to firmware zbudowano: survey-in, Time Mode, korekta piły `qErr`. Dyscyplinowanie zadziała też na tanich modułach nawigacyjnych — w tym na chińskich klonach u-bloxa z eBay/AliExpress — bo pętli wystarczy impuls 1PPS i zdania NMEA. Ale klony ignorują konfigurację binarną: spodziewaj się `0/4 NMEA sentences disabled`, braku ACK dla ramek CFG i ~15 s dłuższego startu, póki nie wygasną timeouts; nie ma survey-in ani `qErr`, wyświetlacz pokazuje stale liczbowe HDOP, a dodatkowa wędrówka pozycji wbija się w fazę — i to właśnie wchłaniają luźne domyślne limity algo-12. Znany dziwaczek klonów: po nieudanej rundzie konfiguracji niektóre moduły przestają wysyłać dane i wyświetlacz stoi na "acquiring" — lekiem jest wejście w tunel u-center (`T` z baudem modułu, np. `T 9600`) i zwykłe poczekanie do wygaśnięcia: ponowna próba po tunelu przywraca pracę. Zażycie tego wszystkiego jest jednym przełącznikiem: zdefiniuj `GPSDO_FAKE_UBLOX` w `gpsdo_config.h`, a firmware robi tylko próbę baudrate i nic więcej nie wyśle — niezależnie od innych włączonych opcji GPS. Jeszcze nie testowany na sprzęcie (autor nie ma modułu-klonu); raporty z pola mile widziane.

3.3 Kalibracja — najpierw CT, potem LC. Kolejność ma znaczenie.

Te dwie komendy uczą firmware dwóch różnych faktów fizycznych, a druga potrzebuje pierwszego:

Krok 1 — CT (czułość oscylatora + strojenie pętli, ~3 minuty). Wpisz `CT` i Enter. Firmware prowadzi napięcie sterujące przez trzy punkty (1,5 V; 2,0 V; 2,5 V), mierzy częstotliwość w każdym, dopasowuje prostą i liczy **K** — **ile herców jest wart jeden krok PWM na Twoim oscylatorze** (akceptowany zakres 0,02–2 mHz/LSB). Z **K** wyprowadza sensowne wzmocnienia PID dla algorytmów 3–9 i pętli LTIC oraz **sam zapisuje grupę PID** do flasha. Nie odcinaj zasilania przez te trzy minuty.

Krok 2 — LC (kalibracja detektora fazy, ~5 minut). Wpisz **LC** i Enter. Firmware zbiori dzielnik picDIV, centruje detektor, przeprowadza jedną rampę kondensatora i mierzy **ns na volt**, **przesunięcie zera w voltach** i **zakres w ns** detektora. Przy **PASS** sam się zapisuje. Odmawia użyteczności, jeśli **CT** nie było uruchomione — dlatego kolejność ma znaczenie.

Kontrola: wpisz **LL** i sprawdź, że blok LTIC ma niezerowe **ns_per_volt**, **zero_offset**, **range_ns**. Wpisz **DAC** — powie Ci nawet, ile mikroherców wart jest jeden krok wyjścia.

3.4 Wybór algorytmu

- **LA 12** — akumulator wielopoziomowy (wg Alana Cashina). Sprawdzony wybór: trzyma fazę na kilku ns RMS, koryguje średnio co kilka minut. **To jest rekomendacja.**
- **LA 11** — ciągła pętla PI Larsa Waleniusa. Dojrzały, łagodny klasyk; znakomite długoterminowe zachowanie z jednym pokrętelem (LTC).
- **LA 13** — filtr Kalmana (część 4.6). Najnowszy: decyduje, ile uwierzyć każdemu odczytowi, z wariacji, a nie ze stałej, i bierze każdą potrzebną liczbę z **CT** i **LC**. Nie ma czego stroić. **Symulator go przemilczał** — na prawdziwym stole ogranicza go wolna wędrówka zera własnego detektora i na metryce odczytu fazy wypada za algorytmem 11 (patrz 4.6a, dlaczego ta metryka może być wobec niego niesprawiedliwa i jakie pytanie stoi otworem). Nowszy niż 12, mniej godzin na sprzęcie — rekomendacja wyżej zostaje; 13 jest tym, na który warto patrzeć.
- **LA 10** — trójstopniowa pętla fazowa (ACQ→DPLL→LOCK). Solidna, więcej parametrów.
- Algorytmy 0–9 to kolekcja historyczna — działają, są zamrożone, patrz Część 4.

LA bez argumentu pokazuje bieżący algorytm. Sam wybór **nie zapisuje się** — patrz krok 3.6.

3.5 Zalecane dodatki

- **SAW 1** — korekta piły. Błąd kwantyzacji impulsu 1PPS odbiornika ($\pm 8... \pm 25$ ns, raportowany co sekundę jako **qErr**) jest odejmowany od mierzonej fazy. Przy odbiorniku timingowym to darmowa dokładność; włącz.
- **TZ <miasto>** — czas lokalny z DST (np. **TZ Warsaw**, **TZ Adelaide**), albo **T0 1** dla stałego przesunięcia, **LT 1** żeby pokazywać czas lokalny zamiast UTC. **H TZ** tłumaczy format reguły, gdyby Twojej strefy zabrakło.
- **P0 <Pa>** — offset dodawany do odczytu ciśnienia BMP280 (–5000..5000).
- **A0 <m>** — offset dodawany do **wysokości z GPS** na wyświetlaczu i w teledymetrii (–3000..3000 m). Koreguje pokazywaną wysokość do realnej rzeźby, np. gdy model geoidy myli się o 30 m: **A0 30**.

3.6 Zapis wszystkiego — ES

Wpisz **ES** i Enter. To zapisuje **wszystkie** ustawienia plus dane wyuczone do flash ringa w sektorze 7. Od tej chwili zanik zasilania niczego nie zabiera.

Garść komend-preferencji zapisuje się **sama** w chwili ustawienia i mówi Ci o tym — odpowiedź zawiera **[auto-saved: ...]**. Są to: **TZ**, **T0**, **LT** (grupa strefy czasowej), **WU**, **SPL**, **SV** (flagi), **P0**, **A0** (offsety), a ponadto **CT** (sam zapisuje strojoną właśnie grupę PID) i zdany **LC** (zapisuje się do pierścienia live).

Wszystko, co dotyczy **pętli** — wzmocnienia (KP/KI/KD/IL), wszystkie parametry LTIC i Larsa, ustawienia algo-12, a nawet sam wybór **LA** — żyje w RAM dopóki nie zapiszesz. Każda taka komenda odpowiada podpowiedzią w stylu **[not saved – run 'ES LTIC' to keep it]**: wykonaj wskazany zapis grupowy, albo po prostu zapamiętaj: **po sesji strojenia — ES**.

3.7 Jak wygląda prawidłowa praca

- Słowo trendu (wyświetlacz + linia **PWM**: raportu) dochodzi do **LOCK** i tam zostaje — przy algo 12 krótkie błyski **CORR/ZC** są **normalne i zdrowe** (korekta to autotest algorytmu wg harmonogramu; wyświetlacz i tak liczy je jako zablokowane).
- Faza (**dph**: w raporcie) błędzi wokół zera w dziesiątkach ns i zawsze wraca — nie może rampować w dal.
- Okna częstotliwości się domykają: **100s**: w milihercach, **1ks**: w mikrohercach, w godzinę–dwie.
- **CS** (statystyka korekt) wypisuje małe, stabilne wartości RMS, które nie rosną z godziną na godzinę.
- Na TFT duża liczba częstotliwości jest **zielona**.

Na spokojnym stole z algo 12 oczekuj: faza 5–20 ns RMS, korekty po kilka LSB co kilka minut, dewiacja Allana w klasie $1e-11 \dots 1e-12$ od 1000 s w górę. Przy grzejniku, otwieranych drzwiach albo pełnym słońcu liczby będą gorsze — to fizyka, nie błąd.

Część 4 — Czternaście algorytmów (0–13) i ich parametry

Jedna idea leży u podstaw wszystkiego: firmware liczy częstotliwość OCXO licznikiem bramkowanym (TIM2), mierzy fazę GPS-vs-OCXO detektorem LTIC i koryguje napięcie sterujące („DAC” PWM, 65536 kroków, $\sim 48,8 \mu\text{V}$ na krok przy 16 bitach; z `GPSDO_PWM_DITHER` efektywna rozdzielczość to 24 bity — około $0,2 \mu\text{V}$ na krok). Konwencja znaku: mierzony błąd $e = \text{freq} - 10 \text{ MHz}$; gdy $e > 0$, oscylator jest szybki i PWM musi iść **w dół** (dla EFC o dodatniej czułości; LPOL / obsługa polaryzacji obejmuje odwrócone EFC).

Jeśli faza, błąd częstotliwości albo PID są dla Ciebie nowymi słowami — zatrzymaj się i przeczytaj Aneks A (za czym pędzi pętla i dlaczego musi być delikatna) oraz Aneks B (co naprawdę robią litery P, I i D). Dziesięć minut tam czyni każdy parametr poniżej czytelnym.

Linia `Learn`: w raporcie nazywa aktywny algorytm; tuner automatycznie dobiera z niego rodzinę wykresów.

4.0 Menu

#	Nazwa (jak pokazywana)	Jedno zdanie	Status
0	primitive	pierwotny sterownik krokowy André Balsy, cykl 429 s	domyślny po zimnym flashu
1	forced-drift	+1 LSB na 1000 s — charakteryzacja oscylatora	diagnostyczny
2	random-walk	szum ± 1 LSB co 5 s — pomiar podłogi szumu	diagnostyczny
3	FLL-PID-man	PID na średniej 100 s	klasyk
4	PLL-PI-man	PI na fazie, cykl 10 s	klasyk
5	PLL-PID-man	jak 4, z własnym slotem strojenia	klasyk
6	FLL-PID-gen	FLL PID, strojenie genetyczne	klasyk
7	PLL-PID-gen	stary roboczy PLL; jego Kp przechowuje wynik CT	klasyk
8	hybrid-FLL-PLL	sigmoidalne mieszanie 6 i 7 wg wielkości błędu	klasyk
9	NN-MLP	mała sieć neuronowa + wyuczony dryf termiczny w holdover	klasyk
10	LTIC-3stage	maszyna stanów ACQ→DPLL→LOCK na detektorze fazy	linia rekomendowana
11	LTIC-Lars	ciągła pętla PI Larsa Waleniusa	linia rekomendowana
12	multi-level	akumulator wielopoziomowy Alana Cashina	rekomendacja
13	kalman	trójstanowy filtr Kalmana — faza, częstotliwość, starzenie	najnowszy

Algorytmy 0–9 są **zamrożone**: zostały, bo działają i bo slot strojenia algorytmu 7 pełni dodatkowo rolę magazynu wyniku kalibracji CT. Nowy rozwój dotyczy wyłącznie 10/11/12/13.

Wybór: `LA <n>` (trwał przez `ES ALGO`). Świeża jednostka zawsze startuje z algorytmem 0 — wybierz swój raz, zapisz, i już zostaje na zawsze.

4.1 Algorytmy 0–2 (diagnostyczne)

Bez parametrów. 0 koryguje PWM, gdy średnie częstotliwości przekraczają stałe progi. 1 prowadzi PWM liniowo (charakteryzacja EFC). 2 wstrzykuje szum (pomiar podłogi pętli). Do normalnej eksploatacji nie są potrzebne.

4.2 Algorytmy 3–9 (klasyki) — parametry przez KP/KI/KD/IL

`KP <algo> <val> / KI / KD` ustawia wzmocnienia (algorytmy 3–7, 0..100000); `IL <algo> <val>` ogranicznik integratora (algorytmy 3–9, 100..100000). `LP [n]` wypisuje. Wartości domyślne (a także baza, od której CT liczy własne, oparte na pomiarze):

algo	Kp	Ki	Kd	I_LIMIT
3	70,0	0,70	175,0	9000
4	1000	0,020	2,0	7000
5	1000	0,020	2,0	10000
6	205	0,264	14950	13000
7	1000	0,020	2,0	10000
8	—	—	—	13000
9	—	—	—	450

Dodatki: algorytm 8 ma **BC** (punkt przecięcia mieszania, domyślnie 0,024 Hz) i **BS** (skala mieszania, 0,012 Hz) — środek i szerokość sigmoidy decydujące, ile FLC a ile PLL wmieszać przy danym błędzie; algorytm 9 ma **NS** (maksymalny krok, domyślnie 175 LSB). Zapis: **ES PID**. Szczerze: po **CT** nie powinieneś musieć tego ruszać — po to jest CT.

4.3 Algorytm 10 — LTIC trójstopniowy

Maszyna stanów na detektorze fazy: **ACQ** (ściąganie po częstotliwości, do tego centrowanie rampy detektora) → **DPLL** (cykl 2 s, częstotliwość + faza) → **LOCK** (korekty co **LIV** sekund z oszacowania pary okien).

Parametry (wszystkie **ES LTIC**; **LL** wypisuje całość):

Komenda	Znaczenie	Zakres / domyślne
LAT	próg fazy dla ACQ	ns, domyślnie 100
LDT	próg dryfu DPLL→LOCK	1e-13..1,0, domyślnie 5e-10
LIV	odstęp korekt w LOCK	1..600 s, domyślnie 300
AQP/AQI/AQD/AQL	PID stopnia ACQ	0..100000
DPP/DPI/DPD/DPL	PID stopnia DPLL	0..100000
LKP/LKI/LKD/LKL	PID stopnia LOCK	0..100000
LPOL	polaryzacja PWM→faza, -1/0/+1	0 = auto
LCV	cel centrowania ACQ	0 (= auto, środek zakresu) .. 3,3
ACG g [cap]	wzmocnienie [limit kroku] napędu centrowania	50..20000 LSB/V [5..1000 LSB]
FA/FAD/FAL	okno uśredniania częstotliwości dla członu tłumiącego (oba / DPLL / LOCK)	10, 100 albo 1000 s

ltic_autotune (część CT/LC) wypełnia wzmocnienia stopni na podstawie zmierzonej czułości, więc opisana sekwencja startowa już ten algorytm stroi. Jeśli ACQ nigdy nie wychodzi, sprawdź **LPOL** — pętla ostrzega i czeka, aż ustawisz.

4.4 Algorytm 11 — LTIC-Lars, ciągła pętla PI

Pętla Larsa Waleniusa, portowana wiernie. Jeden regulator PI liczony co sekundę, z adaptacyjnym pre-filtrem; lock, gdy przefiltrowana faza mieści się w **LPL** ns przez **LTC × LPF** sekund. Detektor na krańcu → automatyczne ściągnięcie po częstotliwości, jedno dozwolone ponowne zbrojenie picDIV.

Komenda	Znaczenie	Zakres / domyślne
LG	wzmocnienie pętli. 0 = auto z CT (zalecane)	0..10000
LD	tłumienie	>0..1000, domyślnie 3
LTC	stała czasowa pętli — jedyne istotne pokrętko	1..600 s, domyślnie 60
LFD	dzielnik pre-filtra (pre-filter = LTC/to)	1..100, domyślnie 2
LTO	cel fazy na detektorze	0..3,300 V, domyślnie 2,111 V

Komenda	Znaczenie	Zakres / domyślne
LPL	okno fazy dla locka	1..10000 ns, domyślnie 100
LPF	współczynnik trzymania locka (hold = LPF × LTC)	1..100, domyślnie 5
LTK	sprężenie termiczne, kroki DAC na krok ADC	-32000..32000, 0 = wyłączone
LTR	odniesienie temperatury	0..3,300 V

Strojenie w jednym zdaniu: zostaw LG 0, ustaw LTC na to, jak delikatnie pętla ma prowadzić (60 s to dobry start; 240 s uspokaja zakłócone miejsce; krócej to tylko więcej szumu), a resztę zostaw domyślnie. Wszystko przez ES LTIC. Zakładka LTIC-Lars w tunerze wystawia dokładnie te pola.

4.5 Algorytm 12 — akumulator wielopoziomowy

Port z GPSDO Alana Cashina (MIS42N), dopracowany w v1.04–v1.06. Zero stałego cyklu: o czasie uśredniania decyduje wielkość błędu.

Jak to działa, prostymi słowami. Co sekundę odczyt fazy wchodzi do drabiny akumulatorów. Poziom n uśrednia po $2^{(n+1)}$ sekund — drabina biegnie 2, 4, 8, ... 2048 s (11 poziomów). Na każdym poziomie siedzi para wartości (A = starsza połowa, B = nowsza). Gdy para się domyka, liczone są dwie liczby: **nachylenie** ($B - A$ = błąd częstotliwości na tym przedziale) i **faza ekstrapolowana** ($3B - A$ = faza właśnie teraz). Jeśli faza mieści się w limicie poziomu, para jest sumowana i awansuje o poziom wyżej (podwójne uśrednianie). Jeśli przekracza limit — korekta strzela natychmiast, drabina się resetuje, a korekta rozkłada się delikatnie na span, z którego przyszła (minimum 64 s — „GPSDO chce duży błąd korygować łagodnie”; u Alana podłoga wynosi 16 s, dobrana tak, by korekta w rozgrzewce nie zażądała napięcia sterującego poza zakresem DAC — mniejsza podłoga to większe kroki, większa to wolniejszy powrót fazy). Niezależnie od tego, osiągnięcie poziomu MR (domyślnie 7 = 256 s) wymusza korektę nawet pod wszystkimi limitami — inaczej powolny dryf nigdy nie doczekałby się reakcji.

Każda korekta ma do trzech części i to jest celowe: skorygować zmierzony błąd **częstotliwości**, sprowadzić **fazę** do zera, i nigdy jedno bez drugiego. Potem trik, który Alan uważa za istotę: wykonany celowo slewing jest zapamiętany, a przy pierwszym odczycie fazy przekraczającym zero w oczekiwanym kierunku dokładnie ten slewing jest zdejmowany (ZC, zniesienie w przejściu przez zero) — oscylator zostaje z właściwą częstotliwością i bez błędu fazy.

Test przejścia przez zero jest uzbrajany **tylko po korekcie wywołanej przez limit** — to reguła Alana. Korekta planowa (poziom MR) pada z zegara, przy fazie tam, gdzie akurat jest, więc nie ma przeregulowania, na które trzeba by czekać; a samo ZC się nie uzbraja ponownie, bo ZC jest zniesieniem. Uzbrojenie w którymkolwiek z tych przypadków pozwoliłoby niezwiązanemu przejściu przez zero kilka minut później wyciągnąć krok z nieaktualnego nachylenia.

Trendy właściwe dla 12: WAIT (brak danych) → SYNC (5 s stabilizacji po zbrojeniu dzielnika) → FLL (detektor na krańcu, ściąganie częstotliwości) → NOPH (brak ważnej fazy, PWM zamrożony) → ACQ → LOCK (>16 s prawdziwego spokoju i częstotliwość w bramce; korekty i ZC nie zerują licznika spokoju — one są dowodem zdrowia, nie szumu). Przejściowe błyski CORR / ZC są normalne. NoCT znaczy: wzmocnienie jest auto, ale CT nigdy nie bieгло — uruchom CT. NoPL znaczy: polaryzacja EFC nieustawiona — pętla czeka, aż ustawisz LPOL -1 albo 1.

Parametry (ES ALG012; ML wypisuje wszystko, łącznie z bieżącym szacunkiem szumu 1-sigma):

Komenda	Znaczenie	Zakres / domyślne
MG	wzmocnienie, LSB na ns fazy. 0 = auto z CT	0..10000, domyślnie 0 (auto)
MR	poziom wymuszający korektę	0..10, domyślnie 7 (256 s)
MF	skąd limity per poziom: 0=za MG, 1=tabela zapisana, 2=wzór sigmowy, 3=zmierzony fit	0..3, domyślnie 0
MFT	przy MF 3: docelowe sekundy między korektami szumowymi	0 (=3600) albo 2048..65535 s
MLP n v	jeden wiersz 11-poziomowej tabeli limitów	poziom 0..10, wartość w nanosekundach

Domyślna tabela limitów to tabela Alana (przeskalowana z jego detektora 25 ns na ten ~1 ns). W nanosekundach na span: L0 462, L1 400, L2 331, L3 264, L4 191, L5 164, L6 126, L7 117, L8 108, L9 103, L10 63 ns. **ML** oznacza ją **UNTUNED** (tylko wiersz 128 s pochodził kiedyś ze specyfikacji) — traktuj jako sprawdzony punkt startowy, nie świętość. Jej geneza, słowami samego Alana, to **najgorszy scenariusz odbioru**: liczby policzono dla NEO-6 z anteną w pomieszczeniu, którego 1PPS potrafi mylić o ± 100 ns przez krótkie okresy (średnio ± 30 – 40 ns). Tabela jest celowo luźna — wymierzona w najgorszy odbiornik, z jakim ten projekt może mieć do czynienia, a nie w sprzęt klasy LEA-M8T, który ta konstrukcja zwykle ma.

Które MF wybrać? Wyniki z pola, dwie płytki:

- **Spokojne miejsce (antena na parapecie w domu, stabilna temperatura)**: wzór sigmowy (**MF 2**) albo pomiar (**MF 3**) działa — pętla sama skaluje się i koryguje co kilka minut.
- **Zakłócone miejsce (warsztat z drzwiami, blisko grzejnika, ruch powietrza)**: autoskalowanie może polować na poziomie 0 (limity ciaśniejsze niż realne wędrówki środowiska). Tam luźna tabela Alana (**MF 1**) była dramatycznie stabilniejsza. Jeśli widzisz ciągłe korekty poziomu 0 — przełącz na **MF 1**.

Za oboma wynikami stoi zadeklarowana zasada projektowa Alana, warta poznania przed strojeniem: „*nie chcemy, żeby algorytm działał limitami, chcemy, żeby działał korektami wg harmonogramu w określonych odstępach*”. Korekta z poziomu run (**MR**) ma być koniem pociągowym, a limity tylko siatką bezpieczeństwa na rozgrzewkę i zaniki odbioru — i właśnie dlatego jego tabela jest luźna. Ciasna tabela strzelająca bez przerwy działa według tej filozofii przeciw algorytmowi, nie z nim.

Wzmocnienie (**MG**) należy do **oscylatora**; limity należą do **szumu miejsca** — właśnie dlatego ustawiają je różne komendy i właśnie po to jest **MF**, żeby dało się wyrazić „zmierzone wzmocnienie, ręczne limity”.

W tunerze: zakładka **Multi-level (algo 12)** ma cztery skalary i całą 11-wierszową tabelę limitów z przyciskami Send/Read/List, a wykresy na żywo same przechodzą na rodzinę algo-12 (ph błąd fazy, poziom, licznik korekt, sigma, licznik ZC).

4.6 Algorytm 13 — filtr Kalmana (autorski)

Algorytmy 10, 11 i 12 odpowiadają na to samo pytanie — *ile z dzisiejszego odczytu fazy mam uwierzyć?* — liczbą ustaloną z góry: etapem maszyny stanów, stałą czasową, poziomem w tabeli progów. Ten odpowiada na nie z wariancji, i odpowiada od nowa co sekundę.

Niesie trzy stany — **fazę, częstotliwość, starzenie** — każdy z kowariancją. Wie, jak głośny jest Twój detektor, bo to mierzy, i jak szybko błądzi Twój oscylator, bo to też mierzy — więc waga każdego odczytu jest tym, co te dwie liczby aktualnie mówią. Przy krótkich czasach uśredniania, gdzie detektor jest głośny a OCXO spokojny, opiera się na oscylatorze; przy długich, gdzie OCXO odchodzi, ustępuje GPS-owi.

Ile kosztuje. Trzy stany i pomiar **skalarny**, więc podręcznikowe odwracanie macierzy to jedno dzielenie: około 140 mnożeń z akumulacją i 36 bajtów, raz na sekundę — jakaś mikrosekunda M4F. Twierdzenie, że Kalman wymaga Cortexa-A albo FPGA, dotyczy dwudziestostanowych filtrów GNSS, nie tego.

Holdover wypada za darmo. Stan niesie częstotliwość i starzenie razem z ich kowariancjami, więc utrata fazy nie jest przypadkiem szczególnym: filtr przestaje aktualizować, dalej przewiduje i dalej steruje. Trend pokazuje **HOLD**.

Korzysta z dwóch pomiarów. Fazy z detektora LTIC i częstotliwości z bramkowanego licznika TIM2. To drugie nie jest ozdobnikiem: przy niesynchronizowanym picDIV nie ma poprawnej fazy w ogóle, a filtr mający tylko pomiar fazy nie ma wtedy **żadnego** pomiaru — przewiduje ze stanu wciąż wyzerowanego, a oscylator odchodzi. TIM2 jest też tym, co pozwala sprawdzić sam detektor: faza i częstotliwość to ta sama wielkość zróżniczkowana, więc w oknie faza **musi** przejechać tyle, ile wynosi suma błędu częstotliwości. Detektor, który się nie rusza, choć TIM2 mówi, że musi, nic nie mierzy — pętla uzbraja picDIV i steruje na samej częstotliwości, póki się nie poprawi.

Wszystko wyprowadzone z CT i LC. Nic w nim nie jest stałą zmierzoną na cudzym stole: własny kwant detektora ($\text{ns_per_volt} \times 3,3/4096$) daje zasiew szumu pomiarowego i jego podłogę, pół pasma detektora daje kowariancję na zimnym starcie, pasmo przejechane w jednym horyzoncie daje tę dla częstotliwości, a R/KT^3 zasiewa szum procesu. Po ponownym **CT** albo **LC** liczby idą za tym w ciągu sekundy. Przy starcie filtr wypisuje jedną linię z tym, co ustalił:


```
KAL: from CT/LC res 1.01ns R0 2.52ns Q0 0.000006600 P0 1500ns lim 939LSB arm<0.25Hz
```

Parametry — wszystkie cztery są opcjonalne, a domyślne są tymi, których się używa:

Komenda	Domyślnie	Znaczenie
KR [ns]	θ = mierz	Szum pomiaru. Zero znaczy „zmiierz go z różnic samego detektora o szesnastosekundowym rozstawie” — biały szum i wolna wędrówka zera razem, i tego właśnie chcesz. Ustawiaj tylko, żeby unieruchomić filtr na czas eksperymentu.
KQ [v]	θ = adaptuj	Szum procesu, $(ns/s)^2$ na sekundę. Zero znaczy „adaptuj z ciągu innowacji”.
KT [s]	100	Horyzont fazy: jak szybko sterowanie zeruje estymatę fazy. Krócej — mocniej za GPS, dłużej — bardziej na oscylatorze. Ustawia też cierpliwość testu zawieszenia: pięć horyzontów daleko od zera i picDIV zostaje uzbrojony.
KL	—	Wypisuje stan: fazę, częstotliwość i starzenie, jak mocno w każde wierzy, aktualne R i Q oraz ile odczytów odrzuciła bramka innowacji.

KR, KQ i KT zapisują się w chwili wpisania, we własnym rekordzie flash ringu — bez ES, a starszy firmware po prostu nigdy o ten rekord nie pyta.

Linia Learn wygląda tak:

```
Learn: algo=13 (kalman) ph=-12.4ns f=-118.30ps/s sig=2.48ns R=2.64 rej=3 arm=1
```

— faza i częstotliwość, w które filtr **wierzy** (a nie odczyt z tej sekundy, bo o to właśnie chodzi w posiadaniu filtru), jak mocno wierzy w fazę, zmierzony szum pomiaru, ile odczytów odrzuciła bramka innowacji i ile razy uzbroił dzielnik. HOLD=<s> pojawia się, gdy pętla jedzie na samym modelu.

W tunerze: górny wykres pokazuje estymatę fazy z filtru, a pod nią v_{phase} z przewodnicami pasma — bo pytanie, które ta pętla stawia najczęściej, brzmi: czy detektor w ogóle żyje.

Zmierzono na symulatorze (tools/loopsim, pięć ziaren szumu, dwie plansze, biały szum detektora): sd fazy 0,81 / 1,20 ns, wobec 3,07 / 4,20 dla algorytmu 12 i 3,62 / 7,39 dla algorytmu 11. **Stół się nie zgadza i stół ma rację:** replay tych samych planów z wolną wędrówką zera odwraca kolejność (11: 7,45 → 10,07 ns; 13: 1,22 → 7,23), a na prawdziwym sprzęcie 29–30.08 ta sama płytka dała 2,96 ns (11) wobec 5,2–5,9 ns (13) z płaską podłogą na średnich czasach uśredniania — poziom wędrówki własnego detektora, niezależny od pasma pętli. Szum symulatora był zbyt biały; traktuj jego liczby jako względne, nigdy bezwzględne (4.6a).

4.6a Jak przebiega sekunda pętli — i co poruszają pokrętła

Jedna sekunda, po kolei. *Predykacja:* stan (faza, częstotliwość, starzenie) przepychany o sekundę. Potem najwyżej dwa pomiary korygują — faza z detektora LTIC (chyba że na szynie, zamrożona lub poza pasmem) i częstotliwość z TIM2 (zawsze; utrzymuje pętlę przy życiu, gdy detektor jest ślepy). Na koniec *sterowanie:* $u = -(\text{freq} + \text{faza}/T)$ — skasuj estymowany błąd częstotliwości i domknij fazę po horyzoncie T, z ograniczeniem do pasma detektora; reszta sub-LSB idzie na następną sekundę. To, co doszło do pinu, wraca do stanu częstotliwości. Wokół rdzenia: bramka 4σ , test zaufania (czy faza rusza tak, jak każe TIM2?), startowe uzbrojenie referencji i holdover.

Insight uczciwego R. Szum detektora mierzony z własnych różnic o szesnastosekundowym rozstawie widzi szum biały (~2,5 ns) i wolną wędrówkę zera (~5,9 ns) razem, $R \approx 6,5$ ns. Ta liczba to cały charakter pętli: dlatego algorytm 13 odmawia gonienia za wolną strukturą detektora, za którą algorytm 11 idzie bez pytania. Zmierzono: przy spokojnej pętli dph pokazuje płaską podłogę 5–9 ns od 10 s do 600 s uśredniania **niezależnie od pasma pętli** — szerokie, adaptowane i sztywne lądują na tym samym poziomie, czyli poziomie wędrówki detektora. Czy gonienie tej wędrówki (11), czy odmawianie (13) daje lepsze wyjście, rozstrzygnie tylko wzorzec niezależny.

KR [ns] (domyślnie 0 = mierz). Przypięcie do podłogi białej (KR 2.5) każe filtrowi podążać za detektorem w pełni: sd@600 poprawiło się ~30% w spokojnych godzinach, ale bramka 4σ zawęży się z R, więc zdarzenia GPS (dziesiątki ns)

są odrzucane — odrzucenia wzrosły ze 163 do 5021 na noc. Przypinaj tylko do obłożenia eksperymentu; `KR 0` jest lepszym domyślnym.

`KQ [v]` (domyślnie 0 = adaptuj). Adaptacja widzi innowacje zabarwione własnym sterowaniem, więc **Q raketuje w górę** — zmierzone 195× seed po jednej nocy (clamp na 1000×). Znane i tolerowane: adaptowana pętla przechodziła nocne epizody GPS wyraźniej lepiej. Przypięcie na seedzie (`KQ 0.000006359` — dokładny seed z linii `KAL: from CT/LC`) usztywnia pętlę ~14× i niemal zamraża PWM; w zmierzonym A/B nic na średnich tau, epizody odrobinę gorzej. **KQ zapisuje się od wpisania** — wróć przez `KQ 0`.

`KT [s]` (domyślnie 100). Jak szybko sterowanie domyka fazę: krócej — mocniejsze trzymanie GPS (i więcej kopiowanego szumu detektora), dłużej — oparcie na stabilności oscylatora. To też cierpliwość czujnika zastoju — pięć horyzontów daleko i picDIV jest uzbrajany. Na stanowisku z wędrówką detektora rozstrzyga, ile z niej dochodzi do oscylatora.

KL — **odczytaj stan** przed, w trakcie i po każdym eksperymencie: estymaty, sigma wiary w fazę, R i Q w użyciu (Q vs seed = prędkość rakiety), ostatnia innowacja, licznik odrzuceń. Najbardziej informacyjny nawyk: `KL`, potem `SW`, potem godzinny log.

Trendy: `KAL` (normalna), `REJ` (bramka odrzuciła), `ARM` (uzbrojenie dzielnika — start, szyna lub zastój), `HOLD` (brak fazy, sterowanie z modelu), `NoPL/NoCT` (brak kalibracji), `WAIT` (brak danych).

Część 5 — Wyświetlacze: co znaczy każde pole

Kilka wyświetlaczy może działać naraz (I2C + SPI nie kłóć się). Wszystkie pokazują tę samą prawdę, z różnym poziomem detalu.

5.1 TFT (480x320 albo 320x240)

```
y= 0..23 pasek nagłówka: "GPSDO v1.06" CPU 58% LMT 14:32:45 Thu
```

`CPU 58%` na pasku nagłówka to całkowite obciążenie procesora, zawsze włączone (pomiar biegnie w zadaniu uptime; `TL` na linii serialowej dokleja rozbieżności per task). Pojawia się od drugiej sekundy po bootcie – okno uśredniania 100 s nie ma jeszcze werdyktu.

```
y= 30..62 FREQUENCY – wielkie cyfry, kodowane kolorem
```

```
y= 70..151 siatka informacyjna, dwie kolumny
```

```
y=156..195 rząd czujników
```

```
y=204..239 pasek statusu
```

Kolor wielkiej częstotliwości to kontrola zdrowia jednym spojrzeniem:

Kolor	Znaczenie
zielony	zablokowany (dla 10/11: trend LOCK; dla 12 również podczas CORR/ZC — korygująca pętla to zdrowa pętla)
biały	dostraja się
pomarańczowy	holdover
czerwony	brak sygnału / fixa

Podczas procedur startowych liczba jest zastępowana pomarańczowymi odliczaniem: Survey <s> ±<m>, OCXO warmup <s>, Tune <s> (CT), LTIC cal <s> (LC), Calibrate <s>.

Siatka informacyjna: kolumna lewa — czas UTC + dzień tygodnia, data, uptime (liczony z 1PPS GPS, więc nie dryfuje), Algo: n <trend>, kod PWM: + napięcie Vct: . Kolumna prawa — liczba Sat: + HDOP: (albo HDOP: TIME po survey-in — to słowo jest *dobrze*, znaczy tryb czasu na stałej pozycji), Lat:/Lon: (6 miejsc), Alt: + qErr: (wartość pięty odejmowana właśnie teraz), INA: napięcie i prąd zasilania. Rząd czujników: BMP: temperatura + ciśnienie, AHT: temperatura + wilgotność, Vph:/dph: napięcie detektora i faza w ns (ovf = rampa poza ważnym pasem), Vcc:/Vdd: szyny zasilania.

Pasek statusu (kolor tła): zielony DISCIPLINED FIX OK, pomarańczowy HOLDOVER (manual), czerwony HOLDOVER (fix lost) albo WAITING FOR GPS FIX; doklejone SURVEY w trakcie survey-in.

5.2 OLED (128x64) — dwie strony, przełączane co 10 s

Strona A (GPS): czas lokalny, częstotliwość, Lat/Lon/Alt+Sats, uptime, UTC+temperatura AHT, PWM + trend (migający H/A na prawej krawędzi = holdover ręczny/auto). Strona B (czujniki): rzędy BMP/AHT/INA, Sat+HDOP, UTC. W trakcie procedur wiersz częstotliwości pokazuje F SVIN <s>s <m>m, F WARMUP <s>s, F CAL <s>s.

5.3 LCD 20x4

Linia 0: częstotliwość; linia 1: UTC + dni uptime; linia 2 rotuje co 10 s (współrzędne / sats+HDOP / AHT / INA / BMP); linia 3: PWM + Vctl + trend z migającym znacznikiem holdover.

5.4 Cyfry zegarowe (TM1637 / HT16K33)

Czas lokalny HH:MM (dwukropki pulsują sekundą). Same kreski = start/brak danych; oooo = brak fixa; spinery = rozgrzewka / survey / kalibracja.

5.5 Diody na płycie

LED	Znaczenie
Żółta (PB8)	WYł = brak fixa GPS · świeci = fix OK · wolne miganie (1 s) = ręczny holdover · szybkie miganie (200 ms) = fix utracony, auto holdover
Niebieska (PC13)	tylko awaria — szybkie miganie znaczy złapaną usterkę firmware (asercja / przepełnienie stosu / brak pamięci), z powodem na serialu. To <i>nie</i> jest pulsometr; ciemna niebieska LED to stan dobry.

Część 6 — Telemetria serialowa (raport 6-liniowy)

Raz na sekundę (raz na PPS) firmware wypisuje blok statusu w tym kształcie tryb human-readable, RH):

```
Up: 000d 02:15:33 UTC: 22/8/2026 14:32:45
Lat: 51.477928 Lon: -0.001531 Alt: 46.5m Sat:10 HDOP:TIME
Freq: 1000000.0000 Hz 10s:0.0 100s:0.02 1ks:0.000 10ks:0.0000
PWM:44653 Vctl:1.970V hit
Learn: algo=11 (LTIC-Lars) gain=auto scale=46 phase=12.3ns LOCK qErr=-8.2ns
BMP:23.4C 1013.2hPa AHT:22.1C 45.3%rH INA:12.05V 250mA Vphase:3.077V dph:1390.5ns CPU:31%
```

(Pozycja w przykładzie to Królewskie Obserwatorium w Greenwich — długość zerowa z definicji, celowo miejsce publiczne. Twoja jednostka będzie oczywiście drukować własne współrzędne; jeśli udostępniasz logi, pamiętaj o opcji **Redact position** w tunerze.)

Linia po linii:

Li ni a	Pola
1	uptime (liczony z PPS, godny zaufania od v1.06) + data/czas UTC z GPS
2	pozycja (6 miejsc), wysokość (wysokość GPS plus Twój A0), satelity, HDOP — albo słowo TIME, gdy odbiornik timingowy jest w trybie stałej pozycji; brak fixa → GPS: no position fix yet
3	suwa liczona częstotliwość (albo --- przed pierwszym zliczeniem) i uśrednione okna błędu 10 s / 100 s / 1 ks / 10 ks — każde pojawia się dopiero, gdy jego bufor się wypełni
4	kod PWM 16-bit, zmierzone napięcie sterujące, słowo trendu (hit, ACQ, DPLL, LOCK, PLL, CORR, ZC, NOPH, NoCT, ARM, NoPL, ...) — w holdoverze [HOLDOVER] zastępuje trend
5	linia Learn zależna od algorytmu (niżej) + qErr, gdy działa SAW 1
6	temperatura/ciśnienie BMP280 (surowe + Twój P0), temperatura/wilgotność AHT, napięcie/prąd INA, napięcie detektora vphase, faza dph w ns (z odjętą piłą, tak jak w samej pętli) oraz CPU: — udział ostatniej sekundy, którego procesor nie spędził beczynnie. Przy TL 1 idzie za tym pole TL: z rozbiciem na zadania.

Linia Learn per rodzina:

- algo 11: gain=auto|<wart> scale=<n> phase=<ns> LOCK|acq
- algo 12: ph=<ns> level=<n> corr=<n> arm=<n> sig=<ns> zc=<n> secs=<n> — nagromadzona faza, ostatni działający poziom, korekty, zbrojenia, bieżący szacunek szumu, przejścia przez zero, sekundy
- algo 13: ph=<ns> f=<ps/s> sig=<ns> R=<ns> rej=<n> arm=<n>, a w holdoverze HOLD=<s> — estymaty filtru, nie odczyt z tej sekundy
- algo 10: state=ACQ|DPLL|LOCK
- algorytmy 3–9: wyuczony dryf/nachylenie/tłumienie, algo 9 dokleja wyuczony tempco

Tryb tab-delimited (RD) drukuje jedną linię na sekundę z tymi samymi danymi jako kolumnami (uptime, okna częstotliwości, sats, HDOP, PWM, napięcia, wszystkie czujniki, surowy TIC) — format do logowania do arkusza. RP/RR pauzuje/wznawia. Raport jest nieblokujący: host, który podłączy się i nie czyta, straci ogony raportów, a nie zamroży wyświetlaczy (naprawione w v1.06 — pełna kolejka CDC potrafiła kiedyś zatkać task wyświetlacza na dobre).

Część 7 — Spis komend (wszystkie)

Serial, 115200, bez rozróżniania wielkości liter, zatwierdzany Enterem. Komendy z argumentem [val] **pokazują bieżącą wartość, wywołane bez argumentu**. Obowiązują dwa reżimy zapisu — firmware zawsze powie w odpowiedzi, na który właśnie trafiłeś:

- **Preferencje zapisują się same i drukują** [auto-saved: ...]: TZ, TO, LT, WU, SPL, SV, PO, AO — a ponadto CT (sam zapisuje wyprowadzoną grupę PID) i LC (sam zapisuje się przy PASS).
- **Parametry pętli żyją w RAM do czasu zapisu** — wszystko, czym obraca tuner: wzmocnienia, parametry LTIC/Lars/algo-12, LA, SP. Odpowiedź wymienia dokładną grupę do utrwalenia zmiany ([not saved – run 'ES ...']).

Wersja, pomoc, stan

Komenda	Co robi
V	wersja, autorzy, podziękowania — oraz CRC-32 obrazu flash , liczone przy starcie z samej pamięci. W odróżnieniu od stempla kompilacji nie może być nieaktualne: builder Arduino używa ponownie plików obiektowych, więc szkic, którego nie ruszałeś, zachowuje starszą datę. Gdy log i pamięć się nie zgadzają, wierz CRC.
H / ?	lista komend · H TZ = szczegóły stref czasowych
SW	znaki wody stosów, wolny heap, uptime + jego źródło, ppm MCU przeciw GPS — oraz obciążenie procesora per task , od największego, uśrednione w prawdziwym oknie 100 s ze stu jednosekundowych kubeków. Mierzone licznikiem cykli Cortexa-M4 przy każdym przełączeniu kontekstu, więc dokładne, a nie próbkowane. Czas przerwań przypada temu zadaniu, które zostało przerwane — czytaj to jako „procesor był tutaj”, nie „to zadanie tyle zużyło”.
TL 0\ 1	te same liczby per task na linii telemetry, jako pole TL: . Po każdym starcie wyłączone i nigdzie nie zapisywane — to diagnostyka przy stole, nie ustawienie.
(bra k)	RP / RR pauzuje i wznawia telemetry — jednoklawiszowe TAB/ESC było próbowane i usunięte: prawdziwe terminale i tuner wysyłają linie zakończone CR/LF i nigdy goły TAB ani ESC, więc z narzędzi, z których ludzie faktycznie korzystają, było nieosiągalne.

Wyjście (DAC)

Komenda	Zakres	Co robi
SP [n]	1..65535 (bez argumentu = 32767, środek ≈1,65 V)	ustawia DAC sterujący wprost — ręczne sterowanie przy eksperymentach
up1/up1 0/dp1/d p10	—	nudge PWM ±1/±10 (odmawiane w trakcie kalibracji)
DAC	PWM/DITH/EXT	bez argumentu: raport — ścieżka wyjścia, kody 24- i 16-bit, Vctl zadane i zmierzone , jeden krok w µHz (wymaga CT). Z argumentem: wybiera aktywną ścieżkę wyjścia — zapisuje się samo . Sygnał przełącza zworka na płytce; to mówi firmware'owi, którą ścieżką steruje. Nieustawione = DITH.

Kalibracja

Komenda	Czas	Co robi
C	~2 min	starsze dwupunktowe centrowanie PWM
CT	~3 min	czułość oscylatora K + autostrojenie PID 3–9 i LTIC; sam zapisuje PID — uruchom najpierw to
LC	~5 min	kalibracja detektora fazy (ns/V, offset, zakres); sam zapisuje się przy PASS — uruchom jako drugie

Komenda	Czas	Co robi
ACG g [cap]	—	napęd centrowania ACQ: wzmocnienie 50..20000 LSB/V, maks. krok 5..1000 LSB
AP	—	ręczne zbrojenie dzielnika picDIV

Tryb i raportowanie

Komenda	Co robi
RH / RD	raport human-readable / tab-delimited
RP / RR	pauza / wznowienie raportu 1 Hz
TL 0\ 1	obciążenie CPU per task na linii telemetry (sw pokazuje je raz; belka TFT pokazuje całkowite CPU% zawsze)
MH / MD	holdover (zamroź PWM, leć sam) / dyscyplina
F	przepłucz bufory pierścieni uśredniania częstotliwości
T [baud]	przezroczysty tunel GPS na USB pod u-center, 300 s (baud 4800..921600)

Wybór algorytmu i klasyczne PID (patrz Część 4)

Komenda	Zakres	Co robi
LA [n]	0..13	wybiera / pokazuje algorytm pętli
LP [n]	algo 0..9	wypisuje parametry PID
KP/KI/KD n val	algo 3..7, 0..100000	ustawia wzmocnienia
IL n val	algo 3..9, 100..100000	ogranicznik integratora
BC / BS	0,0001..1,0 Hz	punkt przecięcia / skala mieszania algo 8
NS	1..10000 LSB	maksymalny krok algo 9

LTIC (algo 10) — zapis przez ES LTIC, lista przez LL

Komenda	Zakres / domyślne	Znaczenie
LNV	0..1e6	nachylenie detektora, ns na volt (mierzy LC)
LZO	0.3,3 V	przesunięcie zera detektora, w voltach
LRN	0..1e9	zakres detektora, ns — uwaga: ta komenda ustawia zakres detektora; znaczenie „włącz/wyłącz samouczenie” z helpu jest w v1.06 nieosiągalne (patrz uwagi)
LAT	0,001..1e9 ns (100)	próg fazy ACQ
LDT	1e-13..1,0 (5e-10)	próg dryfu DPLL→LOCK
LIV	1..600 s (300)	odstęp korekt w LOCK
AQP/AQI/AQD/AQL, DPP/DPI/DPD/DPL, LKP/LKI/LKD/LKL	0..100000	PID stopni
LPOL	-1 / 0 / +1	polaryzacja PWM→faza (0 = auto)
LCV	0.3,3 V	cel centrowania ACQ
FA/FAD/FAL	10 / 100 / 1000 s	okno uśredniania dla członu tłumiącego (oba / DPLL / LOCK)

LTIC-Lars (algo 11) — zapis przez ES LTIC

LG (0..10000, 0=auto), LD (domyślnie 3), LTC (1..600 s, domyślnie 60), LFD (1..100, domyślnie 2), LTO (0..3,300 V, domyślnie 2,111 V), LPL (1..10000 ns, domyślnie 100), LPF (1..100, domyślnie 5), LTK (± 32000 , 0=wył.), LTR (0..3,300 V) — znaczenia w Części 4.4.

Algo 12 (akumulator wielopoziomowy) — zapis ES ALG012, lista ML

MG (0..10000 LSB/ns, 0=auto z CT), MR (0..10, domyślnie 7), MF (0.3 źródło limitów, domyślnie 0), MFT (0=3600 s, albo 2048..65535 s), MLP <poziom> <ns> (poziom 0..10) — znaczenia w Części 4.5.

Algo 13 (Kalman) — zapis w chwili wpisania, bez ES

KR (0..1000 ns, 0 = mierz), KQ (0..1, 0 = adaptuj), KT (10..10000 s, domyślnie 100), KL (wypisz stan filtru) — znaczenia w części 4.6.

GPS, czas, czujniki

Komenda	Zakres	Co robi
SV 0\ 1	—	survey-in / Time Mode wył./wł. (działa od następnego startu) — zapisuje się samo
TZ <miasto\ reguła>	np. TZ Adelaide	strefa czasowa z DST (H TZ po szczegóły) — zapisuje się samo
TO <h[:mm]\ A>	-14..+14	stałe przesunięcie UTC, albo A = auto z pozycji GPS (reguła DST unijna) — zapisuje się samo
LT 0\ 1	—	pokazuj UTC / czas lokalny — zapisuje się samo
PO <f>	-5000..5000 Pa	offset ciśnienia dodawany do odczytu BMP280 — zapisuje się samo
AO <f>	-3000..3000 m	offset wysokości dodawany do wysokości GPS — zapisuje się samo
SAW 0\ 1	—	korekta piły (qErr) wył./wł. — samo SAW pokazuje stan; zalecane Wł. przy odbiorniku timingowym (zapis: ES FLAGS)
WU 0\ 1	—	rozgrzewka OCXO przy starcie — zapisuje się samo
SPL 0\ 1	—	animacja startowa wył./wł. — zapisuje się samo

Zapis, odczyt, reset

Komenda	Co robi
ES [obj]	zapisz wszystko (bez argumentu) albo jedną grupę: TZ, PID, LTIC, FLAGS, ALG012, ALGO, PO
ER	przyciągnij — wczytaj ustawienia z flasha teraz (cofnij niezapisane zmiany)
EE	wymaż slot ustawień — domyślne od następnego startu
EW	zużycie flash ringa: cykle wymazań, zajęte sloty, sektor/adres
FR	status pierścienia tylko do odczytu (zawsze włączony od v0.96)
CS	statystyka korekt: liczba, szczyt, RMS ostatnich 100/1k/10k/100k korekt — małe i stabilne jest dobrze, rosnące jest źle
RB	ciepły restart (zachowuje ustawienia — ale nie zapisuje ich najpierw sam)
CR YES	zimny restart: wymaż ustawienia + dane wyuczone, domyślne fabryczne (YES jest wymagany, bo model wyuczony odbudowuje się dniami)

Znane osobliwości v1.06 (uczciwa lista)

- Linia helpu H dla ES nie wymienia ALG012 — sama komenda ją przyjmuje (ES ALG012 zapisuje blok algorytmu 12).

Część 8 — Tuner na PC

`tools/gpsdo_tuner.py` — konsola desktopowa do oglądania i strojenia. Robi wszystko, co terminal, a do tego wykresy na żywo i logowanie CSV.

8.1 Instalacja i start

Python 3.9 lub nowszy, potem:

```
pip install PySide6 pyqtgraph pyserial tzdata
```

(`tzdata` używane tylko przez przycisk *Generate tz_table.h*; na Linuksie/macOS system to ma.)

Uruchomienie: `python gpsdo_tuner.py` (na Windows można też dwuklikiem).

8.2 Połączenie

Wybierz port na pasku, baud 115200, Connect. Tuner natychmiast czyta wersję firmware i **wszystkie** bieżące parametry (LL, FA, LP 3–LP 9, wszystkie czasowniki Larsa i algo-12, ML) i wypełnia każdą zakładkę — widzisz rzeczywisty stan urządzenia, nie domyślne. Linia stanu pokazuje state: LOCK (locked) itd.; tytuły wykresów podążają za aktywnym algorytmem.

8.3 Zakładki

Zakładka	Co edytuje
LTIC (algo 10)	trzy kwartety PID stopni; Read (LL) / Save (ES LTIC) / Revert (ER)
LTIC-Lars (algo 11)	LG LD LTC LFD LTO LPL LPF LTK LTR z przyciskami Set
Multi-level (algo 12)	MG MR MF MFT + cała 11-wierszowa tabela limitów (Send limits, Read all, List (ML), Save (ES ALG012))
FA damping	okna tłumienia DPLL / LOCK
PID algo 3-9	Kp/Ki/Kd/IL per algorytm
Calibration	LNV LZO LRN LCV LAT LIV LPOL
Raw monitor	wszystko, co mówi płytka, bez parsowania; sterowanie logowaniem
Help	referencja komend firmware

Pamiętaj, że reguła firmware obowiązuje i tu: przyciski Set zmieniają tylko RAM; przycisk Save zakładki wysyła ES ..., który utrwała.

8.4 Wykresy

Trzy panele, odświeżane ciągle z telemetrii 1 Hz. Górne panele zależą od rodziny algorytmu — faza (dph/ph) + napięcie detektora dla 10/11/12 (z szarą kotwicą i czerwonymi krawędziami ważnego pasma, gdy kalibracja jest znana), dryf + napięcie sterujące dla klasyków. Dolny panel to zawsze błąd częstotliwości w Hz. Pasek: wybór zakresu czasu (1 min ... all), Follow live, Clear. Rozdzielczość to szybkość telemetrii — zdarzenia szybsze niż ~2 s są niewidzialne, a wykresy ufają płytce.

8.5 Logowanie (wykresy to nie logger)

Raw monitor → Start logging, format Full log / CSV only / Both:

- Full log `gpsdo_YYYY-MM-DD_HH-MM-SS.log` — każda linia jak wypisana, ~217 MB na tydzień przy 1 Hz.
- CSV — sparsowane kolumny: utc, up_s, algo, state, dph_ns, qerr_ns, vphase_v, pwm, f10, f100, ph_ns, level, corr, sig_ns, zc, bmp_c, sat, hdop (~65 MB/tydzień). Pusta komórka = pola tej sekundy nie było (nigdy zero).

- **Redact position** (domyślnie WŁ.) czyści Lat/Lon/Alt w zapisanym pełnym logu — dziel się logami bez publikowania własnego dachu.

Część 9 — Ustawienia, flash ring i zużycie

Wszystko, co trwałe, mieszka w jednym **pierścieniu z wyrównywaniem zużycia** we flash sektorze 7: 255 slotów po 512 bajtów. Każdy zapis (**ES**, zdany **LC**, aktualizacja danych wyuczonych) pisze kolejny pusty slot; sektor jest wymazywany dopiero, gdy pierścień zawija się — raz na 255 zapisów. Przy tempie zapisów tego firmware (nawet zapracowany stule robi ~73 zapisy dziennie) wymazanie wypada co ~3,5 dnia, a flash F411 znosi ~10 000 cykli na sektor: **mniej więcej 96 lat**. Nie zużyjesz tego. **EW** pokazuje realne liczniki, gdybyś był ciekawy.

Dwa rodzaje rekordów dzielą pierścień:

- **Ustawienia** (**ES** i spółka) — wszystko, co wybrałeś, zapisywane tylko na Twoje polecenie. Zapisy grupowe (**ES** **PID** itd.) piszą *cały* blok ustawień wyseedowany ostatnim zapisanym — więc zapis jednej grupy nie może nadpisać innej.
- **Dane live** — to, czego urządzenie nauczyło się samo: kalibracja **LC**, model dryfu/tłumienia, ostatni punkt pracy. Zapisywane automatycznie, ale z histerezą (tylko gdy coś realnie zmieniło się o sensowny krok, i co najmniej co 20 minut).

Aktualizacje firmware: ustawienia z v1.04/v1.05 są migrowane przy przywołaniu (v4→v5 dokładają blok algo-12 z domyślnymi); blok z radykalnie innej wersji albo uszkodzony slot traktowany jest jak „nieobecny” — domyślne i prośba o kalibrację przy następnym starcie. Sam pierścień się leczy: slot na wpół zapisany przez zanik zasilania nie przechodzi CRC i jest po prostu pomijany; wygrywa poprzedni dobry.

Praktyczna checklista, ostatni raz:

- rutynowe wgrywanie: bezpieczne, ustawienia przeżywają (Część 2),
- po sesji strojenia: **ES**,
- przed sprzedażą/podarowaniem jednostki: **CR YES**,
- gdy jednostka dziwnie się zachowuje po sesji eksperymentów: **ER** (przywołaj zapisane, znane-dobre ustawienia), a dopiero potem diagnozuj.

Część 10 — Diagnostyka problemów

Objaw	Prawdopodobna przyczyna → rozwiązanie
Nie kompiluje się, błąd 1toa	rdzeń STM32 3.0.0 — zjedź do 2.12.0 (Część 1.1)
Kompiluje się, wypisuje ? albo śmieci zamiast liczb	Tools → C Runtime Library → Newlib Nano + Float printf/scanf
TFT biały po buildzie na rdzeniu 3.0.0	to samo — rdzeń 2.12.0
TFT biały na rdzeniu 2.x	zły sterownik w User_Setup.h albo brak TFT_MISO PA6 (Część 1.4)
Start wisi zaraz po TFT: init start	jak wyżej — sprawdź okablowanie i #define sterownika
Brak seriala USB po wgrywaniu	naciśnij RESET; jeśli dalej nic — sterownik Zadig dla VID 0483/PID 5740 (Część 2.5)
Ustawienia/kalibracja zniknęły po wgrywaniu	ktoś użył Erase Chip — to zasada sektora 7 (Część 2.2); powtórz CT, LC, ES
Żółta LED wyłączona	brak fixa GPS — antena, widok nieba, kabel
HDOP:TIME się nie pojawia	survey-in nieukończony (wymaga odbiornika timingowego + sv 1 + dobrego nieba); powtarza się po każdym zaniku zasilania
CT kończy się porażką / odrzuca K	okablowanie EFC albo czułość oscylatora poza zakresem 0,02–2 mHz/LSB — sprawdź bufor DAC i zakres EFC
LC kończy się porażką	najpierw CT; nie ruszaj jednostki przez te 5 minut
Trend stoi na NoCT (algo 12)	wzmocnienie auto, ale CT nigdy nie biegło → CT
Trend stoi na ACQ / NoPL, algorytmy 10–13	nieustawione LPOL (pętla drukuje ostrzeżenie i czeka) — ustaw LPOL -1 albo 1, potem ES LTIC
Algo 12 koryguje bez przerwy na poziomie 0	zakłócone miejsce — MF 1 (tabela Alana), patrz Część 4.5
Faza rampuje w dal po SAW 1	nie masz odbiornika timingowego / qErr nieważny — SAW 0
Linie raportu zamierają, gdy program otwiera CDC i nie czyta	naprawione w v1.06 (zapisy nieblokujące + kolejka 1 KB); jeśli widzisz — działasz na starym firmware
Niebieska LED szybko miga	złapana usterka firmware (asercja / stos / heap) — przeczytaj komunikat na serialu; zgłoś
Jednostka sama się resetuje	sprawdź szynę 3V3 pod obciążeniem OCXO; baner startowy drukuje przyczynę resetu
Wyświetlacz żyje, serialu brak	jesteś tylko na Bluetooth? 57600 na Serial2; albo wpisano RP — RR wznowia

Jeśli nic z tego nie pomaga: złap pełny log tunerem (Część 8.5), zanotuj wersję z **V** i pytaj — z dołączonym logiem. Tydzień CSV-a 1 Hz to różnica między zgadywaniem a wiedzą.

10.1 Jak zgłosić problem

Prawie każde pytanie o to firmware dało się rozstrzygnąć czterema rzeczami, a bez nich rozstrzyga się je dniami zgadywania zamiast minutami czytania. Wyślij wszystkie cztery:

1. **Log bootowania** — wszystko od resetu do `GPS init done`, skopiowane jako tekst, nie sfotografowane. Mówi, jaka to płytką, jaki baud GPS wykryto, które czujniki się odezwały, które ramki UBX zostały potwierdzone i jaka była przyczyna resetu.
2. **Twój skompilowany `gpsdo_config.h`** — albo choćby lista przełączników, które zmieniłeś. Większość niespodzianek to różnice konfiguracji, nie usterki: płytką bez detektora fazy, drugi wyświetlacz na tych samych pinach, `SAW 1` na odbiorniku nawigacyjnym.
3. **Jaki moduł GNSS** — prawdziwy u-blox timingowy, prawdziwy nawigacyjny, czy klon (Część 3.2). Klony zachowują się inaczej i sama ta informacja od razu odcina połowę możliwości.
4. **Antena i to, ile widzi nieba**. „W domu na parapecie” to całkowicie dobra odpowiedź i często całe wyjaśnienie.

Jedno sprawdzenie przed wysłaniem: w banerze musi być linia `compiled <data> <godzina>`. Jeśli jej nie ma, masz build sprzed v1.05 i pierwszą rzeczą do spróbowania jest aktualny — kilka zgłoszeń okazało się błędami już naprawionymi. Tę linię drukuje samo firmware, więc nie może się rozjechać z binarką tak, jak zapamiętany numer wersji.

Jeśli płyta **zawiesza się**, dopisz jedną darmową obserwację: czy niebieski LED na PC13 dalej mruga? Przełącza go przerwanie timera 2 Hz i nie zależy od żadnego tasku, więc mruganie znaczy, że MCU żyje, a zablokował się task — to zupełnie inna usterka niż płyta, która w ogóle nie działa.

Aneks A — Jak działa GPSDO, prostymi słowami

Możesz używać urządzenia bez tego aneksu — ale gdy coś zacznie dziwnie zachowywać się, to te dwadzieścia akapitów powie Ci *dlaczego*.

A.1 Problem

Chcesz sygnał 10 MHz, który jest prawidłowy **teraz** (sekunda po sekundzie) i prawidłowy **zawsze** (po miesiącach i latach). Dwa cegiełki dostarczają po połowie i żadna obu:

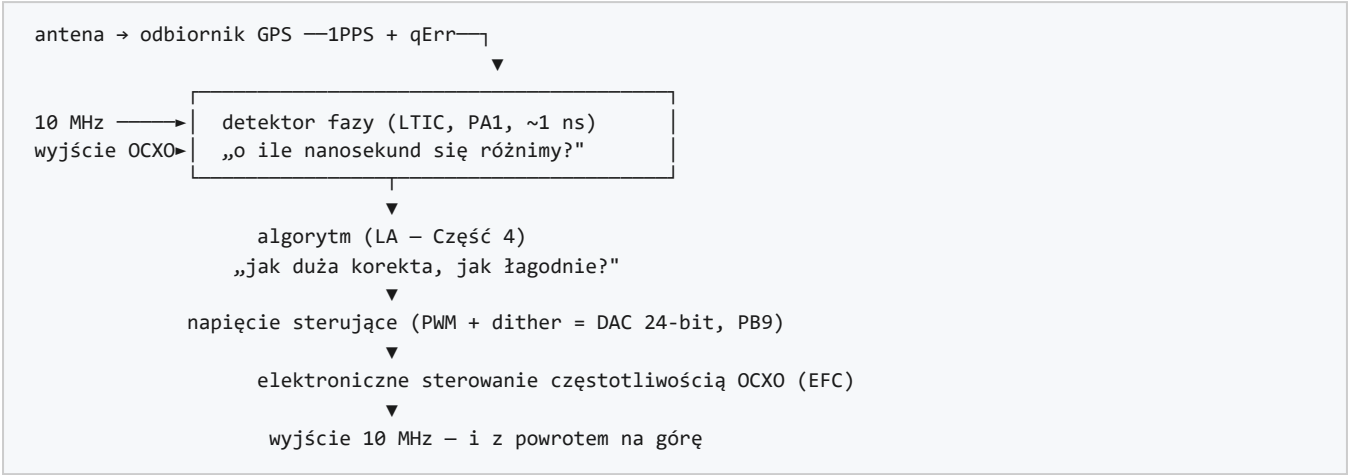
	Krótko (sekundy)	Długo (miesiące)
Sam OCXO	znakomity — dobry trzyma części 10^{11}	dryfuje — starzenie i temperatura powoli odciągają częstotliwość
Sam 1PPS z GPS	szumy — każdy impuls ląduje w $\sim \pm 25$ ns od prawdy	znakomity — sterowany zegarami atomowymi, bez końca

Część „piecu” w nazwie OCXO to pół historii: kryształ siedzi w podgrzewanym pudełku trzymanym na jednej stałej temperaturze (to ta 300-sekundowa rozgrzewka przy starcie), bo temperatura to największy wróg kryształu. Zostaje starzenie — powolny spacer, którego nikt nie wyłączy.

Impuls GPS jest odwrotnością: każda pojedyncza sekunda jest tylko przybliżona (odbiornik kwantyzuje czas, sygnał zwalnia w jonosferze, odbija się od budynków), ale *średnia* z godzin jest przybita do czasu atomowego, bo satelity niosą zegary atomowe i GPS bez tego nie działa.

GPSDO to podział pracy: OCXO obsługuje sekundy, GPS obsługuje miesiące, a pętla sterująca pomiędzy nimi przesuwa tę pierwszą, małutkimi krokami, trzymając ją przypiętą do tej drugiej.

A.2 Pętla, element po elemencie — i gdzie jest co na tej płytce



- **Detektor fazy** odpowiada „o ile spóźnieni jesteśmy?” raz na sekundę, w nanosekundach. Ta jedna liczba — *faza* — to tablica wyników, na której toczy się cała gra.
- **Licznik (TIM2)** odpowiada „jesteśmy szybcy czy wolni?”, licząc dosłownie 10 MHz. Jest gruby (kroki 0,01 Hz), ale bezpośredni.
- **Algorytm** patrzy na te liczby i decyduje jedno: o ile podrobić napięcie sterujące i na jakim czasie rozłożyć szturchnięcie.
- **DAC:** PWM o 65536 krokach, ditherowany do efektywnych 24 bitów (kroki $\sim 0,2 \mu V$ — oscylator reaguje na mniej niż jeden krok PWM, więc firmware pamięta ułamek między krokami).
- **Nóżka EFC oscylatora:** wolty na wejściu, herce na wyjściu — cały zakres regulacji ma ledwie kilka herców szerokości, i właśnie dlatego mikrovolty mają znaczenie.

A.3 Faza, częstotliwość i jedyna wymnażanka, jakiej potrzebujesz

Błąd częstotliwości i ucieczka fazy to ten sam fakt w dwóch kostiumach. Jeśli oscylator odjeżdża o ułamek, jego faza ucieka dokładnie w tym tempie:

Błąd częstotliwości przy 10 MHz	Faza ucieka
1 Hz	100 ns co sekundę
0,01 Hz	1 ns co sekundę
0,001 Hz (1 mHz)	1 ns co 10 sekund

Dwie konsekwencje warte wyrzucia:

1. Jeśli faza się nie rusza, częstotliwości są równe — niezależnie od tego, jaka jest wartość fazy. Prawdziwy cel pętli to *nieruchoma* faza, zaparkowana blisko zera.
2. Błędy, o które warto walczyć, są absurdalnie małe. Jeden miliherc przy 10 MHz to jedna część na 10^{10} — a pętla rutynowo rozróżnia dziesięć razy lepiej. Dlatego każdy przewód, każdy mikrowolt i każdy stopień mają tu znaczenie większe niż w każdym innym obwodzie, który budowałeś.

A.4 Dlaczego korekty muszą być delikatne (serce całego tematu)

Wyobraź sobie, że impuls GPS przyszedł 20 ns za późno, bo sygnał odbił się od budynku. Zmierzony „błąd” fazy to 20 ns — ale nie jest realny: oscylator się nie ruszył. Jeśli pętla skoryguje natychmiast po wartości nominalnej, **skopiuje szum GPS na oscylator** i wyjście będzie *gorsze* niż swobodnie biegnący OCXO. Ten jeden błąd to cała różnica między GPSDO a oscylatorem młóconym przez GPS.

Obrona to uśrednianie: losowy szum maleje jak $\sqrt{N}$. Uśrednij 100 sekund $\rightarrow$ szum $\div 10$; 1000 s $\rightarrow \div 31$. Pętla czeka więc, aż uśrednione liczby z GPS staną się cichsze niż własny dryf oscylatora, i dopiero wtedy działa — korektą wymierzoną w to, co długie uśrednienie realnie udowodniło, rozłożoną na porównywalny czas. Dlatego:

- algorytm 11 ma LTC (stałą czasową — „jak cierpliwa jest ta pętla”),
- algorytm 12 pozwala wielkości błędu wybrać własny czas uśredniania (duży błąd jest udowadniany w sekundy i w sekundy korygowany; mały — przez godzinę i przez godzinę),
- każda korekta w tym firmware jest ditherowana w kroki sub-LSB zamiast wylewania się naraz.

Jeden darmowy obiad: **korekta piły (SAW 1)**. Odbiornik wie, o ile skwantyzował każdy impuls (liczba `qErr`) i wyznaje to co sekundę. Odjęcie tego wyznania zamienia ± 25 ns fałszywego błędu w kilka ns, zanim w ogóle zacznie się jakiegokolwiek uśrednianie.

A.5 Holdover

Gdy GPS znika (przecięta antena, martwy odbiornik), pętli brakuje prawdy, którą mogłaby prowadzić. Właściwy ruch to **zamrożenie** ostatniego dobrego napięcia sterującego i pozwolenie OCXO płynąć na własnej stabilności — to holdover (MH ręczny albo automatyczny po utracie fixa; żółta LED miga, wyświetlacz robi się pomarańczowy, [HOLDOVER] zastępuje trend w raporcie). Oscylator dryfuje wtedy ze starzenia i temperatury — powoli, ale niepowstrzymanie, dopóki GPS nie wróci.

A.6 Jak ocenia się wyniki: dewiacja Allana w jednym akapicie

Standardy częstotliwości porównuje się **dewiacją Allana (ADEV)**: mniej więcej „o ile średnia nie zgadza się z samą sobą, brana po τ sekund”, kreślona przeciw τ . Krzywa GPSDO opada od lewej do prawej: przy $\tau = 1$ s widać własny szum oscylatora; gdy τ rośnie, uśrednianie zbija szum i przejmuje kotwiczenie z GPS (na stole autora: $\sim 3 \cdot 10^{-9}$ przy 1 s, spadając do $\sim 5 \cdot 10^{-12}$ przy 3000 s). Gdy ktoś pisze „1e-12 przy tau 10 000”, to właśnie to zdanie mówi. *Garb* na środku krzywej znaczy, że pętla walczy sama z sobą — za szybko jak na własny szum — i to klasyczny podpis źle strojonego GPSDO.

Aneks B — PID dla opornych

Nigdy nie *potrzebujesz* tego aneksu, żeby używać urządzenia — CT stroi pętlę za Ciebie. To na dzień, w którym otworzysz zakładkę **PID algo 3-9** w tunerze albo wpiszesz **KP** i zechcesz wiedzieć, którą dźwignię trzymasz i w którą stronę gryzie. Nie używa się matematyki ponad mnożenie.

B.1 Obrazek prysznic

Każda pętla sterująca w historii robi te same cztery rzeczy:

- 1. **Zmierzyć**, gdzie jesteś (temperatura wody).
- 2. **Porównać** z tym, gdzie chcesz być (komfort).
- 3. Różnica to **błąd** (za zimno o 5 stopni).
- 4. **Zadziałać** (odkręć gorący kran), odczekać, powtórzyć.

Jedyny temat całej dziedziny to krok 4: *o ile odkręcić kran?* PID — trzy litery, trzy odpowiedzi — to standardowa recepta.

B.2 P — proporcjonalne: „reaguj na teraz”

Odkręć kran proporcjonalnie do błędu. Zimno mocno → odkręć mocno. Zimno lekko → odkręć lekko.

- **Samo P ma wadę**: potrzebuje pewnego błędu, żeby w ogóle produkować jakiekolwiek działanie, więc osiada *blisko* celu, nigdy na celu — ostatni stopień zimna nie wystarcza, by utrzymać kran otwarty. Ta resztką to **droop**.
- **Za dużo P**: przestrzeliwujesz na gorąco, potem na zimno, i oscylujesz między nimi — prysznic z piekła.
- **Za mało P**: dojeżdżasz wiekową.

B.3 I — całkowite: „pamiętaj przeszłość”

Patrz na błąd *w czasie*. Nawet maleńkie uporczywe zimno kumuluje się, w sumie bieżącej, dopóki pętla nie doda tyle działania, by domknąć. **I zabija droop** — to człon „docelowo, dokładnie”.

- **Za dużo I**: klasyczne powolne jo-jo. Suma narasta w podejściu, potem wydaje się przestrzałem, odbudowuje się z drugiej strony i pętla huśta się leniwie w nieskończoność.
- **Windup**: gdy kran jest już pełniutko otwarty (wyjście na ograniczeniu), suma rośnie bezużytecznie i potem wieki się rozkręca z powrotem. Lekarstwo to ogranicznik sumy — i dokładnie tym jest **IL** (I_LIMIT).

B.4 D — różniczkowe: „antycypuj”

Reaguj nie na błąd, tylko na to, jak *szybko* się zmienia. Szybko się ociepla? Zaczynaj przykręcać *zanim* dojedziesz. **D to tłumik** — zawieszenie, które krzyżuje oscylacje, które P i I inaczej by pokochały.

- **Wada D**: wzmacnia szum pomiaru (różniczkowanie drgającego czujnika daje iglice). Dlatego człony D są zwykle filtrowane, małe, albo jedno i drugie — i dlatego szumiący pomiar fazy psuje strojenie obciążone D, a nie poprawia.
- W algorytmach PLL (4/5/7) slot **Kd** działa na nagromadzoną *fazę*, a nie na surową pochodną — ta sama służba tłumienia, inne okablowanie. Nie daj się zmylić literze; myśl „pokrętko tłumienia”.

B.5 Trzy litery, jedna tabela

Pokrętko	Reaguje na	Leczy	Za dużo →	Za mało →
P	błąd właśnie teraz	ospałość	przestrzał, szybka oscylacja	dojazd wiekową
I	błąd nagromadzony w czasie	stojący offset (droop)	powolne jo-jo, windup	osiada blisko, nigdy na celu
D	tempo zmian błędu	przestrzał (tłumienie)	nerwowy dryf za szumem	przestrzał dzwoni

B.6 Gdzie siedzą pokręta w tym firmware

Kręcisz	Gdzie	Którą to naprawdę literą
KP n val / KI / KD / IL n val	algorytmy 3–9 (zakładka PID algo 3-9)	dosłownie P, I, D i ogranicznik integratora
AQP...AQL, DPP...DPL, LKP...LKL	trzy stopnie algorytmu 10 (zakładka LTIC)	pełny zestaw PID na stopień — pętla dostraja się sama w miarę uspokajania
LG / LD / LTC	algorytm 11 (zakładka LTIC-Lars)	wzmocnienie (jak mocno), tłumienie (jak osiada), stała czasowa (jak cierpliwie) — te same trzy pokręta w ubraniach Larsa
MG i limity	algorytm 12 (zakładka Multi-level)	brak PID — patrz niżej

Algorytm 12 zasługuje na jeden uczciwy akapit: **nie ma P, I ani D**. Zamiast stałych wzmocnień pyta za każdym razem „jak duży jest błąd, jak długo uśredniałem, żeby go udowodnić?” i skaluje korektę do pary pytanie-odpowiedź — duże błędy dostają szybkie, stanowcze traktowanie; małe dostają powolne, delikatne. To ta sama fizyka z wbudowanym strojeniem — dlatego jego jedyne ręczne pokręta są wzmocnienie (MG, a CT mierzy je za Ciebie) i limity rozstrzygające, co liczyć za „duże”.

B.7 Dlaczego istnieje CT — problem linijki

Liczby PID żyją w jednostkach „kroków PWM na herc błędu”. Ale jeden krok PWM jest wart *inną liczbę herców na każdym egzemplarzu oscylatora* — 0,32 mHz na jednym z Vectronów autora, prawie siedem razy mniej na budowie z wąskim EFC. Bez zmierzenia Twojego oscylatora to samo $K_p = 1000$ jest delikatnym muśnięciem na jednej płytce i gwałtownym pchnięciem na innej. CT mierzy odpowiedź volt-na-herc Twojego oscylatora i przeskala każde wzmocnienie do pary — tak, że fabryczne strojenie znaczy to samo fizycznie na każdej jednostce. Dlatego kolejność kalibracji w tej instrukcji to prawo: najpierw CT, potem LC, strojenie (jeśli kiedykolwiek) na końcu.

B.8 Dziesięć zasad kciuka

1. Uruchom CT i LC zanim dotkniesz jakiegokolwiek wzmocnienia. W większości żywotów to koniec strojenia.
2. Zmieniaj jedno pokrętko naraz.
3. Potraj albo dziel przez dwa — nigdy $\times 10$. Strojenie pętli reaguje na stosunki, nie na arytmetykę.
4. Daj każdej zmianie godzinę. Te pętla uśredniają po minutach; ocena po trzydziestu sekundach to czytanie książki po jednej literze.
5. Wątpliwości → wolniej (większe LTC, mniejsze wzmocnienia). Wolno znaczy nudno; szybko znaczy niestabilnie.
6. Oscylacje → tnij P, potem I.
7. Offset, który nigdy do końca nie domyka → więcej I — albo, bardziej prawdopodobnie, pominąłeś CT.
8. Wyjście widocznie goni szum GPS → wolniejsza pętla, SAW 1, lepszy widok anteny. Nie większe tłumienie.
9. ES po każdej sesji; ER od-experymentowuje zły popołudnie.
10. Jeśli walczy z Tobą cały dzień — podejrzewaj sprzęt przed strojeniem: widok nieba anteny, temperaturę (grzejniki, drzwi, słońce), okablowanie EFC. Pętla w tym firmware trudno zepsuć, a łatwo obwiniać.

Aneks C — Słowniczek

Pojęcia, które przewijają się przez tę instrukcję, telemetrię i changelog — w znaczeniu, w jakim używa ich ten projekt. Sugestia Alana Cashina, który zwrócił uwagę, że połowa z nich gdzie indziej znaczy co innego.

Pojęcie	Co znaczy tutaj
ADEV	Odchylenie Allana — standardowa miara stabilności oscylatora: o ile zmienia się względna częstotliwość między sąsiednimi oknami uśredniania o długości τ . Czyta się to jak „przy 100 s ten zegar trzyma 1e-11”.
ACQ / DPLL / LOCK	Trzy etapy algorytmu 10. ACQ ściąga częstotliwość blisko, korzystając z licznika; DPLL szybko domyka fazę i częstotliwość; LOCK poprawia rzadko i wąskopasmowo, dochodząc do minimum błędu.
BBR	Pamięć podtrzymywana bateryjnie w module GNSS — tam zapisana konfiguracja odbiornika przeżywa zanik zasilania, jeśli jest ogniwo podtrzymujące.
DAC	Przetwornik cyfrowo-analogowy: to, co zamienia liczbę na napięcie sterujące. W tej konstrukcji zwykle nie jest to żaden układ scalony — patrz PWM i dither.
kod DAC / LSB	Liczba zapisywana na wyjście napięcia sterującego i jeden jej krok. Wszystkie wzmocnienia podajemy na LSB, bo to najmniejszy ruch, jaki pętla może wykonać.
dither	Celowe zmienianie młodszych bitów wypełnienia PWM z okresu na okres, tak by <i>średnia</i> wypadła między dwoma krokami sprzętowymi. 13 bitów sprzętowych odtwarzanych z tablicy 2048 pozycji daje około 24 bity efektywne.
EFC	Elektroniczne strojenie częstotliwości — wejście strojące OCXO. Wolty na wejściu, herce na wyjściu.
holdover	Praca bez GPS: pętla przestaje korygować, a OCXO biegnie swobodnie na ostatnim napięciu sterującym.
LTIC	Licznik odstępów czasu Larsa — rampowy detektor fazy na PA1 (kondensator ładowany między zboczem PPS a zboczem podzielonego OCXO). Samo „TIC” znaczy ten sam detektor.
NMEA	Tekstowe zdania wysyłane przez moduł GNSS (\$GPRMC,...): pozycja, czas, liczba satelitów — wszystko poza konfiguracją binarną.
OCXO	Oscylator kwarcowy w termostacie: kwarc trzymany w stałej temperaturze — dlatego jest stabilny i dlatego wymaga rozgrzewki.
picDIV	Mały dzielnik, który z 10 MHz robi zbocze 1 Hz dla detektora fazy. Trzeba go <i>uzbroić</i> (zresynchronizować), by jego zbocze padało blisko PPS.
PID / PI	Regulator: P reaguje na błąd teraz, I na błąd nagromadzony, D na to, jak szybko się zmienia. Większość pętli tutaj to PI — patrz Aneks B.
PPS	Wyjście jednego impulsu na sekundę z odbiornika GNSS — wzorec czasu, względem którego mierzy się wszystko.
PWM	Modulacja szerokości impulsu: fala prostokątna, której wypełnienie po filtrze RC staje się napięciem sterującym. Tanio, a z ditherem lepiej niż większość układów DAC.
qErr / piła	Własne oszacowanie odbiornika, w pikosekundach, o ile jego zbocze PPS spudłowało prawdziwy czas. Odbiorniki timingowe je podają (UBX-TIM-TP); korekta usuwa błąd o kształcie piły.
survey-in	Odbiornik timingowy przez długi czas mierzy własną pozycję, by potem trzymać ją na sztywno i cały wysiłek obliczeniowy poświęcić <i>czasowi</i> .
Time Mode	Stan, w który odbiornik timingowy wchodzi po survey-in: pozycja ustalona, timing zoptymalizowany. Wyświetlacz pokazuje HDOP: TIME.
trend	Czteroznakowe słowo w telemetrii i na wyświetlaczu, nazywające to, co pętla robi w tej chwili: ACQ, DPLL, LOCK, CORR, ZC, NOPH, ...
Vctl / Vphase	Vctl to napięcie sterujące idące <i>do</i> oscylatora; Vphase to napięcie detektora wracające z pomiaru fazy. Dwa różne piny, łatwo pomylić.
ZC	Zniesienie w przejściu przez zero (algorytm 12): zdjęcie celowego slewingu dokładnie w chwili, gdy faza przechodzi przez zero — częstotliwość i faza zostają poprawne naraz.